Topology-Aware Placement of Network and Compute Resources in Multi-Socket Low-Latency Systems

Author metadata pending (non-final placeholder)

Pre-experiment manuscript — August 2026

Abstract

Multi-socket servers expose several non-uniform paths between a network interface and packet-processing code. CPU affinity can stabilize a worker without selecting its packet memory; NUMA binding can place that memory without controlling the network interface card's PCI Express path; and receive steering can assign a hardware queue while leaving descriptor access or inter-stage synchronization remote. Treating these controls independently may therefore produce a locally plausible but globally inconsistent request/response path. This paper formulates their joint placement as a systems and mathematical-modeling problem. We define a typed attributed graph that distinguishes processor, cache, memory, PCIe, network-interface queue, descriptor, packet-pool, event-ring, logical-role, physical-execution-context, and worker objects. Over this graph, we specify a constrained cost model with explicit group scales and a reproducible nonnegative calibration estimator, and a deterministic bounded heuristic that maps complete RX/TX pipelines over finite ownership catalogs while enforcing occupancy, capacity, SMT, and isolation constraints. We also design an automatic configuration architecture based on hwloc, Linux sysfs, PCI bus/device/function identifiers, CPU-affinity interfaces, and NUMA policy with independent read-back. A pre-specified evaluation will compare operating-system defaults, CPU affinity alone, CPU affinity with memory binding, and complete topology-aware placement under one frozen DPDK or AF_XDP profile. The sole confirmatory response is the conditional paired change in nearest-rank run-level p99.9 external turnaround latency among accepted B3/B2 pairs; all policy-attempt and adjacent-pair completion rates are mandatory operational-feasibility outcomes. Correctness, throughput, secondary ablations, and resource counters support interpretation. Results are deferred until measurements on a qualified physical multi-socket testbed are available.

Keywords: NUMA; PCI Express; tail latency; network I/O; resource placement; topology graph

1 Introduction

Low-latency networked software is evaluated not only by the rate it sustains but also by the delay of unusually slow packets or requests. A low mean can coexist with episodes caused by queueing, interference, or resource migration, and such episodes can dominate composed services [6]. Userspace runtimes accordingly report high percentiles, including p99.9, when evaluating bursty microsecond-scale workloads [25]. This paper addresses one source of such variation inside a multi-socket host: inconsistent placement along the path from a network interface card (NIC) hardware queue to the worker that processes the packet.

That path crosses resources controlled by different mechanisms. Receive-side scaling (RSS) maps flows to queues; a driver or userspace framework supplies descriptor rings and packet buffers; direct memory access (DMA) moves data through a PCI Express (PCIe) hierarchy; a polling thread or interrupt context advances the queue; an event ring may transfer ownership to a runtime worker; and CPU affinity and NUMA policy constrain execution and pages. PCIe experiments show that DMA locality and transaction structure affect host networking [24]. Non-uniform DMA is distinct from ordinary CPU NUMA access because the device reaches memory through a particular I/O attachment [33]. Queue steering must additionally respond to flow skew [1]. No one of these controls specifies the complete path.

Consider a dual-socket host whose NIC is beneath a PCIe root associated with socket 0. Pinning the worker to an otherwise idle core on socket 1 may improve CPU isolation. First-touch allocation can then place its packet pool on NUMA node 1, making worker loads local. The NIC, however, must write those

buffers through the inter-socket path. Moving the pool to node 0 makes DMA local but makes the worker's packet reads remote. A producer polling on socket 0 and a worker on socket 1 can also exchange ownership cache lines across sockets on every ring operation. Each isolated choice is defensible; their composition can introduce remote DMA consumption, remote loads, or coherence traffic.

The reviewed literature addresses important subsets. NUMA schedulers coordinate threads and pages while accounting for asymmetric bandwidth or contention [19, 5, 32]. I/O studies characterize PCIe and non-uniform DMA [24, 33]. Packet systems control queue steering, buffering, receive-ring sharing, or runtime scheduling [1, 27, 28, 29]. X-Lane explicitly selects a processing core on the NUMA node associated with a NIC's PCIe attachment, but as part of a specialized jitter-isolation design rather than a general placement model [16]. General topology mapping represents logical and physical communication graphs [13, 17]. Across the 42 archived full texts and the targeted update search documented for this study, we did not identify work that jointly maps RX/TX queues, descriptor and payload memory, event rings, polling or interrupt contexts, runtime workers, processor/cache domains, and the PCIe/NUMA path, then isolates their incremental contribution to p99.9 external latency with the four required baselines. This is a bounded review finding, not a universal absence claim.

The study asks six research questions. **RQ1** estimates CPU affinity relative to operating-system or frontend defaults. **RQ2** asks for the incremental effect of CPU-relative NUMA memory binding after affinity is fixed. **RQ3** asks for the incremental effect of completing the RX/TX descriptor–payload–ring–context–worker and PCIe-aware mapping. **RQ4** examines how these effects vary with packet size, offered load, queue count, flow count, worker count, and burstiness. **RQ5** tests which cost terms explain observed differences through ablation and counter correlation. **RQ6** evaluates stability across workload and topology changes. The sole confirmatory hypothesis is that, among blocks with an accepted B_3/B_2 pair at one pre-specified reference condition, full B_3 placement reduces run-level p99.9 external turnaround latency relative to B_2; RQ1 and RQ2 are key secondary increments. The corresponding null states a nonnegative conditional mean paired log ratio. Policy-specific acceptance and pair-completion rates are reported separately, so the conditional latency comparison is not generalized to all scheduled blocks. The experiment, not the model, determines the decision.

The contributions planned and specified in this pre-experiment manuscript are:

1. a typed attributed graph that preserves distinctions among processor, memory, cache, PCIe, NIC, and software-pipeline objects;
2. a constrained, interpretable placement objective separating topology-derived, calibrated, and workload-dependent costs;
3. a deterministic bounded heuristic mapping NIC RX/TX queues to finite ownership-catalog entries and logical workers, descriptor and payload memory, event rings, physical execution contexts, and PUs, with explicit occupancy, capacity conflicts, completion semantics, complexity, exact-small reference semantics, and incomplete-data fallbacks;
4. a conceptual automatic-configuration design based on hwloc, sysfs, PCI bus/device/function (BDF) identities, affinity and NUMA interfaces, read-back validation, and rollback; and
5. a reproducible four-baseline methodology centered on externally measured p99.9 latency, correctness gates, paired inference, sensitivity studies, and complete run artifacts.

The scope is conventional multi-socket Linux hosts with PCIe NICs and software-controllable queues, memory allocation, and worker affinity. High-frequency trading is a motivating application, but the model targets low-latency packet-processing systems generally. This work does not design a NIC, replace the operating-system scheduler, optimize application semantics, or claim universal benefit from locality. It does not yet implement the proposed components or report measurements. The remainder proceeds from the hardware mechanisms to prior solution boundaries, a formal mapping, an implementable design specification, and a controlled evaluation plan.

2 Background and Motivation

2.1 Processor and memory topology

A physical socket contains one processor package, but the scheduling and memory objects exposed by a platform can have a finer or different structure. A core contains one or more processing units (PUs), commonly corresponding to simultaneous multithreading (SMT) contexts. Caches may be private or shared, and the last-level-cache (LLC) sharing domain need not coincide with a socket. A NUMA node identifies a memory domain with a processor proximity and allocation policy. Modern platforms may expose several NUMA nodes per socket, or a relationship not representable as a one-to-one socket mapping. hwloc therefore models PUs, cores, caches, packages, and NUMA nodes as distinct objects [11, 4].

A local CPU load is served by a memory controller close to the executing CPU; a remote load traverses a socket interconnect such as Intel Ultra Path Interconnect or a platform-specific equivalent. Distance alone is incomplete. Interconnects can be asymmetric, and a nominally local controller or link can be congested. Thread/page placement studies find that bandwidth and traffic pressure can change the preferred mapping [19, 5]. The model must therefore distinguish a static proximity relation from load-dependent capacity and contention.

Huge pages change translation and allocation granularity but do not eliminate physical placement. A packet-buffer pool may span pages allocated under first-touch, preferred-node, bound-node, or interleaved policy. Its intended node and observed page locations are separate facts: policy constrains allocation, whereas read-back establishes the result.

2.2 PCIe and non-uniform I/O

A PCIe endpoint, such as a NIC function, is identified by a domain:bus:device.function BDF. Its path can traverse a switch, bridges or root ports, and a root complex connecting the I/O fabric to processor and memory resources. Link generation, width, payload size, transaction overhead, and path sharing constrain that path. A BDF identifies the endpoint in one topology snapshot; it does not by itself state the cost of reaching every memory or CPU domain. hwloc can report PCI devices and locality, while Linux sysfs exposes device ancestry and NUMA hints [11]. Some root-complex or routing details may be collapsed or absent and must remain unknown rather than inferred as zero cost.

NIC receive DMA writes packet data and queue state to host memory; transmit DMA reads data, and MMIO writes can ring device doorbells. Interrupt-driven paths add interrupt-routing and handler placement, whereas busy-polling userspace paths assign a polling context. IOctopus shows why these I/O operations are a separate form of non-uniformity: an endpoint attached through one processor can reach a remote memory domain through the inter-socket fabric even if no application thread migrates [33]. PCIe measurements likewise show that the location and transaction shape of host buffers matter; descriptor-sized transfers and packet data need not have the same optimum [24]. Intel Data Direct I/O (DDIO), where present, can place incoming data in LLC and introduces cache-capacity and interference effects that also depend on queue load [9].

2.3 NIC queues and packet-I/O frameworks

Modern NICs expose multiple RX and TX hardware queues. RSS hashes selected packet headers and uses an indirection table to steer flows; equal bucket counts do not imply equal work when flows have different rates or processing costs. RSS++ demonstrates the need for load-aware bucket-to-core assignment [1]. Queue progress can be interrupt-driven, kernel-polled, or continuously polled in userspace. Interrupt affinity is relevant only to an interrupt path; assigning it in a pure userspace-polling profile would not control the poller.

The Data Plane Development Kit (DPDK) maps NIC queues into userspace and commonly combines busy polling, huge-page-backed pools, batching, and explicit core assignment. FastClick's cross-framework analysis illustrates how these mechanisms reduce per-packet overhead while exposing configuration choices [2]. AF_XDP retains a Linux driver path and connects an XDP program to userspace through a shared UMEM region and RX, TX, fill, and completion rings. Its copy or zero-copy mode, wakeup behavior, polling configuration, and driver support affect delay [14, 7]. DPDK and AF_XDP are consequently separate

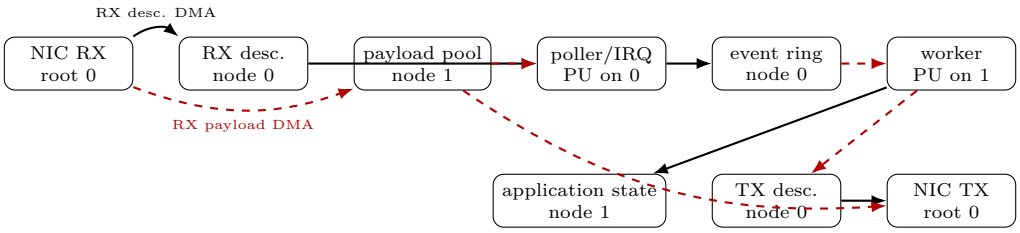


Figure 1: A motivating request/response placement. Solid and dashed edges denote candidate local and cross-socket relations, not measured magnitudes. Independent CPU and memory policies can leave RX/TX DMA, descriptor access, event synchronization, or application-state access remote; the joint model scores the complete path.

experimental profiles. A placement comparison is valid within a frozen profile; switching profile changes more than topology policy.

2.4 Pools, event rings, and workers

A packet-buffer pool owns packet storage and associated metadata; its pages have physical NUMA locations. A hardware descriptor ring is consumed or produced by a NIC and software. An event bus or application ring is a distinct producer–consumer structure used to hand work from a poller to a runtime worker. Shared-memory packet pipelines use such rings to avoid copies [2]. The cache line containing a head, tail, or ownership flag can move between producer and consumer cache domains even when the payload is not copied.

The worker executes application logic on an eligible PU and may access packet data, ring state, and separate application state. In a run-to-completion design, poller and worker are logical roles of one physical thread and occupy one hardware-thread slot while their service demands add; a staged design uses independent physical contexts and a material handoff. Receive rings also combine distinct responsibilities: supplying empty buffers to a NIC and delivering filled buffers to software. ShRing shares receive rings among processes, whereas RxBisect separates the allocation and delivery roles, showing that queue/ring ownership cannot be assumed to be permanently one-to-one [27, 28]. Concurrent single-queue receive and state–compute replication provide further examples in which queue, state, and compute cardinalities differ [8, 35]. The experiment must therefore declare and verify one ownership mode for each profile rather than embed a one-queue–one-worker assumption in the model.

2.5 Motivating inconsistent placement

Figure 1 follows a complete request/response path on a two-socket host. CPU-only pinning fixes the worker but leaves RX/TX descriptors, payload memory, and the event ring subject to unrelated defaults. Adding memory binding can make worker access local while preserving a remote NIC-to-memory path. Joint placement evaluates both DMA directions, both software accessors, the synchronization edge, and the control path; it may choose either socket depending on measured costs and load. X-Lane's decision to place its dedicated core on the NIC's PCIe-local NUMA node illustrates one concrete subset of this problem [16], but the diagram does not assert that one side is universally best.

Application-specific studies already reveal parts of this interaction. A raw-Ethernet storage protocol aligns channel state, cores, memory, and NICs [12]; Maestro evaluates combinations of NIC, packet-buffer memory, application state, and CPU when parallelizing network functions [26]. The remaining question is how to represent, score, and evaluate the complete mapping as a reusable placement problem.

3 Related Work

3.1 NUMA-aware thread and memory placement

NUMA placement work treats thread and page locations as coupled decisions. Lepers et al. show that asymmetric link bandwidth can matter more than hop count and construct a dynamic placement policy around this observation [19]. Carrefour targets traffic congestion through placement, replication, and interleaving rather than treating local access as the only objective [5]. Blagodurov et al. coordinate contention management with NUMA locality because migrating a thread away from its pages can exchange one bottleneck for another [3]. These systems motivate the separation of topology distance, capacity, and observed load in our cost function. Their resource sets end at processors, caches, links, and memory; they do not include a NIC endpoint, hardware queue, DMA destination, or application ring.

Phoenix extends coordinated CPU/page placement with on-demand page migration and replication based on bandwidth and contention [32]. Its published scope remains CPU execution and memory pages: it neither models a NIC's PCIe attachment nor connects RX/TX ownership, DMA destinations, and runtime rings. It therefore strengthens the need for load-aware memory terms without closing the I/O-path gap.

Core-aware runtimes expose physical CPUs directly to applications. Arachne lets applications manage cores rather than opaque kernel threads, but explicitly leaves NUMA and hyperthread issues outside its scope [30]. Caladan reacts to cache, memory-bandwidth, and SMT interference at microsecond time scales [10]. These designs motivate explicit worker capacities and optional re-planning, not the claim that pinning alone establishes an I/O-local path.

3.2 Non-uniform I/O and PCIe/NIC locality

Neugebauer et al. decompose PCIe transactions for end-host networking and measure how buffer location, DMA operation, DDIO, and IOMMU configuration affect behavior [24]. IOctopus identifies non-uniform DMA as an I/O-attachment problem and removes it through multiple coordinated physical functions, one near each processor [33]. Hostping uses active intra-host probes to diagnose root-port, PCIe-switch, memory-channel, and inter-socket bottlenecks, demonstrating that structural and time-varying conditions both matter [21]. DDIO analysis adds cache allocation and interference to the I/O path [9].

These works establish that NIC-to-memory locality is distinct from worker-to-memory locality. IOctopus changes device architecture, Hostping diagnoses rather than plans, and PCIe/DDIO studies characterize mechanisms. The present work instead asks how unmodified, software-controllable resources should be mapped together and verified.

X-Lane is a close end-host precedent: it isolates a dedicated core, configures receive handling, and selects a core on the NUMA node associated with the NIC's PCIe lanes to avoid the inter-socket path [16]. Its objective is bounded jitter for a specialized express lane, with broader OS/network isolation and optional SmartNIC support. It does not formulate alternative pool/ring/worker assignments, a calibrated multi-term cost, or the incremental four-policy experiment required here.

3.3 Userspace packet processing and queue steering

Userspace packet frameworks combine kernel bypass or early driver hooks with batching, huge-page-backed memory, polling, and multi-queue steering. FastClick compares these mechanisms across frameworks [2]. XDP retains kernel integration, while AF_XDP exports UMEM and rings to userspace; driver mode and polling configuration form part of the experimental treatment [14, 7]. Ensō redesigns the NIC–application interface as streams to reduce fixed-buffer and PCIe transaction overhead [31]. These architectures inform the resource types in the graph but do not formulate multi-socket placement.

RSS++ dynamically modifies the RSS indirection table to respond to skew and preserve flow state, and separates a NUMA-aware assignment discussion from its primary load-balancing design [1]. MICA uses key steering and partitioned ownership to align requests, cores, state, and NIC resources for one key-value-store architecture [20]. Maestro coordinates RSS, state, packet buffers, and cores for automatically parallelized network functions [26]. They demonstrate useful subsets of joint placement, but their application semantics, objectives, and reported percentiles do not yield the general queue–pool–ring–worker p99.9 experiment proposed here.

ShRing and RxBisect expose an orthogonal design axis: a receive ring may be shared, and its empty-buffer allocation role can be separated from filled-buffer delivery [27, 28]. State–Compute Replication likewise decouples the number of state or receive resources from compute replicas under an external sequencing design [35]. These works prevent the proposed planner from assuming one queue, one ring, and one owner, but they optimize receive-path interfaces or stateful scaling rather than host NUMA/PCIe placement. A 2024–2025 DPDK development proposal also explores hwloc-backed lcore grouping by cache, SMT, and I/O domains; its mailing-list discussion highlights ambiguity between such domains and DPDK socket identifiers [34]. It is relevant implementation prior art, not peer-reviewed performance evidence.

3.4 Topology-aware mapping and scheduling

hwloc supplies a portable hierarchy of compute, memory, cache, and I/O objects and preserves distinctions that flat socket or NUMA identifiers lose [11, 4]. TreeMatch maps an application communication matrix onto hierarchical compute resources [17]. Hoefler and Snir formulate generic topology mapping using weighted logical and physical graphs, routes, distance, and congestion [13]. This work adopts their separation of demand and physical cost but extends the vertex and edge types to heterogeneous I/O and software resources. A PCIe hierarchy is not merely another set of identical compute nodes: DMA direction, memory eligibility, queue ownership, and worker capacity impose typed constraints.

Cluster schedulers provide complementary formulation patterns. Explicit-cost scheduling can separate preferences from hard capacity, while interference-aware scheduling shows why workload behavior cannot be derived from static machine type alone. Their task-to-machine granularity is too coarse for a single-host packet pipeline, so they inform architecture rather than serve as direct baselines.

3.5 Low-tail-latency runtimes

Shenango uses an IOKernel and rapid core allocation to handle bursty latency-sensitive workloads and evaluates p99.9 [25]. ZygOS uses work-conserving cross-core scheduling for microsecond tasks [29]; Shinjuku adds preemption for dispersed or heavy-tailed service times [18]. Together they establish that queue organization, burstiness, service-time distribution, and load must appear in the workload matrix. They optimize CPU scheduling and runtime queueing, not the physical PCIe/NUMA path feeding those runtimes.

3.6 Experimental methodology

Tail percentiles require sufficient observations, independent repetitions, and uncertainty; a throughput-only study cannot decide a p99.9 hypothesis. External configuration can also bias a systems comparison. Mytkowicz et al. show that seemingly irrelevant layout and environment choices can reverse performance conclusions and recommend setup randomization [23]. The Linux scheduler's documented failure modes further motivate observing actual affinity and progress instead of assuming the default policy behaved as intended [22]. The protocol in section 9 therefore freezes non-placement variables, randomizes policy order within matched blocks, validates each applied assignment, and excludes incorrect runs before looking at performance.

3.7 Synthesis

The literature supplies evidence for each mechanism but not, in the inspected corpus and targeted update search, the complete proposed treatment. CPU/page placement omits the I/O path; I/O studies omit the runtime chain; receive-ring systems alter ownership semantics without selecting the full host path; steering systems omit one or more memory and synchronization decisions; topology mapping lacks typed I/O semantics; and tail runtimes focus on scheduling. The resulting gap is not terminological. We did not identify a reproducible joint RX/TX mapping with the incremental four-baseline p99.9 external-latency test proposed here. The next section defines that mapping without presupposing its empirical benefit.

4 System Model and Problem Statement

4.1 Target system and pipeline

The system under test (SUT) is a cache-coherent, multi-socket Linux host with at least one PCIe NIC. The platform exposes CPU-affinity and NUMA-allocation controls, and the selected packet-I/O profile exposes hardware queues, packet and descriptor memory, and queue-progress ownership. The confirmatory experiment freezes either a DPDK or AF_XDP frontend; it does not compare the frameworks as placement policies.

A finite set $\mathcal{Q}$ contains logical request/response pipelines. The traffic contract fixes for each $q \in \mathcal{Q}$ an RX queue $r_q \in \mathcal{R}^{\mathrm{rx}}$ and compatible TX queue $t_q \in \mathcal{R}^{\mathrm{tx}}$ for the duration of a matched baseline block. A request traverses RX descriptors and payload memory, an applicable polling or interrupt role, an optional event ring, a worker role and optional application state; the response traverses TX payload and descriptors before NIC transmission.

The resource inventories and their cardinalities, (r_q, t_q), and the flow-to-queue rule are common to B_0–B_3. A topology-agnostic reference mapping

$$\mathcal{R}_0 = (\omega_0, \kappa_0^{\mathrm{rx}}, \kappa_0^{\mathrm{tx}}) \tag{1}$$

fixes queue-to-worker relations and direction-specific indices into the finite ownership catalogs defined below for B_0–B_2. The complete B_3 treatment may select ω and controllable catalog indices κ^d jointly with physical placement, subject to the same resource counts, ownership cardinalities, packet semantics, and offered work. Thus an ownership change in B_3 is an explicit finite placement decision, not a change to workload or implementation. Secondary ablations in section 9 separate this logical-mapping effect from physical topology placement.

4.2 Notation and complete placement

Table 1 defines the symbols used below. Calligraphic letters denote finite sets; s identifies a topology snapshot and ℓ a workload condition. A processing unit (PU), core, NUMA node, cache domain, and PCIe root domain remain different objects even when a particular platform reports a one-to-one relationship.

Table 1: Notation for the topology and placement model.

Symbol	Meaning
G_s	Typed attributed topology graph for snapshot s
$\mathcal{Q}$	Logical request/response pipelines
$\mathcal{R}^d$	Hardware queues in direction $d \in \{\mathrm{rx}, \mathrm{tx}\}$; r_q, t_q are the queues fixed for q
$\mathcal{M}^d$	RX/TX payload-buffer pools
$\mathcal{D}^d$	RX/TX descriptor-memory objects
$\mathcal{B} = \mathcal{B}^+ \cup \{b_\varnothing\}$, $\mathcal{A} = \mathcal{A}^+ \cup \{a_\varnothing\}$	Material event rings and application-state regions plus distinguished no-ring and no-state objects
$\mathcal{W}$, $\mathcal{H}$, $\mathcal{L} = \mathcal{W} \uplus \mathcal{H}$	Runtime-worker roles, controllable I/O roles, and their typed disjoint union
$\mathcal{X}$, $\mathcal{C}$, $\mathcal{K}$, $\mathcal{N}$	Physical execution contexts (occupancy groups), PUs, physical-core/SMT groups, and NUMA memory domains
$\nu_X(x)$	Declared location distribution of a material memory object $x \in \mathrm{mat}(X)$, $X \in \{\mathcal{M}^d, \mathcal{D}^d, \mathcal{B}, \mathcal{A}\}$, over $\mathcal{N}$
$\widetilde{\nu}_{bjm}$, τ_{mem}	Observed page-distribution vector for material object m in policy/run (b, j), and the dimensionless frozen B0/B1 total-variation tolerance
$\lambda^d_{q\ell}$, $\lambda^B_{q\ell}$	Direction-d packet rate and event-handoff rate of q in condition ℓ (events/s); $\lambda_{q\ell} = \lambda^{\mathrm{rx}}_{q\ell}$ is the logical request rate used for workload weighting
$a^W_{q\ell}$	Worker service demand of q (PU-seconds/s)
$a^{H,d}_{q\ell}$	Total direction-d I/O-context demand of q (PU-seconds/s)
$\mu^d(q), \xi^d(q)$	Payload pool and descriptor-memory object assigned in direction d
$\varphi(q), \zeta(q)$	Event ring and optional application-state region assigned to q

Symbol	Meaning
$\mathcal{P}_q^d$, $\kappa^d(q)$	Frozen finite ownership catalog for pipeline q and direction d, and its selected catalog index
$O^d(q)$, $\bar{\pi}_{qh}^d$	Owner-role set and dimensionless planning share derived from $\mathcal{P}_q^d[\kappa^d(q)]$; neither is a free continuous decision
$\widetilde{\pi}_{qh}^d$	Observed workload-dependent service share when the frontend cannot enforce an exact share
$\gamma(u), \eta(x)$	Physical execution context used by logical role $u \in \mathcal{L}$ and PU used by active context $x \in \mathcal{X}$
x_{init}	Topology-blind B0/B1 initialization context, excluded from the measured data plane
$c(u) = \eta(\gamma(u))$	Derived PU of an active logical worker or I/O role
$\mathcal{R}_0$	Topology-agnostic reference worker and finite ownership-catalog mapping used by B_0–B_2
σ_{mx}	Predeclared CPU-access demand share of execution context x for material memory object m; shares sum to one over its actual CPU accessors
Π	Complete placement $(\mu^{\text{rx}}, \mu^{\text{tx}}, \xi^{\text{rx}}, \xi^{\text{tx}}, \varphi, \zeta, \omega, \kappa^{\text{rx}}, \kappa^{\text{tx}}, \gamma, \eta)$; O^d and $\bar{\pi}^d$ are catalog-derived
$\mathcal{F}(G_s, \ell)$	Placements satisfying all hard constraints
$J(\Pi; G_s, \ell, \boldsymbol{\alpha}, \boldsymbol{\beta})$	Dimensionless placement score with component weights $\boldsymbol{\alpha}$ and group-level coefficients $\boldsymbol{\beta}$
$\perp$	Unknown attribute, distinct from a measured zero

Each material memory object is location-qualified. For $x \notin \{b_\varnothing, a_\varnothing\}$, the vector $\nu_X(x) \in [0,1]^{|\mathcal{N}|}$ sums to one and records the intended fraction of its pages on each NUMA node; a bound-node object has a one-hot vector. The desired vector and the observed post-fault page distribution are separate records. This representation admits a deliberately interleaved object without calling it "local" to every accessor.

The distinguished objects $b_\varnothing$ and $a_\varnothing$ have no pages and no NUMA distribution. They reserve zero bytes, slots, and service rate and contribute zero topology, synchronization, state-access, uncertainty, and reconfiguration cost. $b_\varnothing$ is eligible only when the frozen pipeline has no material producer–consumer handoff; $a_\varnothing$ is eligible only when the frozen service has no separately placed application state. Both are excluded from material capacity sums. These are formal null objects, not zero-capacity allocations that another pipeline can select.

For each q and direction d, the frontend capability profile supplies a nonempty finite ownership catalog

$$\mathcal{P}_q^d = \{p_{qdi} = (O_{qdi}, \bar{\boldsymbol{\pi}}_{qdi}, m_{qdi}, A_{qdi}) : i \in \mathcal{I}_q^d\}, \qquad |\mathcal{I}_q^d| < \infty, \tag{2}$$

where $O_{qdi} \subseteq \mathcal{H}$, $\bar{\boldsymbol{\pi}}_{qdi}$ is an explicitly enumerated planning-share vector over that set, m_{qdi} is a control-class tag, and A_{qdi} is the predeclared read-back acceptance set. The three tags are DIRECT, VALIDATED-INDIRECT, and OBSERVED-ONLY. The selected finite index $\kappa^d(q) \in \mathcal{I}_q^d$ derives $O^d(q) = O_{qd\kappa^d(q)}$ and $\bar{\pi}_{qh}^d = \bar{\pi}_{qd\kappa^d(q),h}$. The catalog, its stable order, capability evidence, and digest are frozen before calibration and confirmation.

The ownership cardinality k_q^d is a profile invariant. If $k_q^d = 0$, the catalog contains only the null entry $(\varnothing, \varnothing)$ and every corresponding placement and cost term is absent. If $k_q^d = 1$, each admitted entry has exactly one owner and its only share is one. If $k_q^d > 1$, the planner can enumerate only the share vectors explicitly present in equation (2); it never searches a continuous simplex. A DIRECT entry is requested exactly. A VALIDATED-INDIRECT entry may be requested through a documented indirect control but is admitted only when independent read-back lies in A_{qdi}. For an OBSERVED-ONLY entry, the profile freezes one externally supplied or calibration-derived planning vector $\bar{\boldsymbol{\pi}}_{qdi}$ and admissible observation range A_{qdi}; $\widetilde{\pi}_q^d$ is measured at run time, the applicator makes no exact-share claim, and B3 cannot vary the share. Such shares are workload-dependent observations rather than placement controls.

The primary confirmatory profile uses only single-owner entries with $k_q^d = 1$ and $\bar{\pi}_{qh}^d = 1$ whenever an owner role exists. Multi-owner catalogs are secondary or replication profiles until stand qualification establishes a versioned capability record, finite catalog, and reproducible validation gate. This restriction preserves the general model while making the primary candidate space enforceable.

A physical execution context $x \in \mathcal{X}$ is one independently schedulable thread or equivalent controllable entity and consumes one hardware-thread slot. The map γ relates logical roles to these physical occupants, while η places each active occupant on one PU. Distinct roles may share x only when a predeclared co-execution predicate permits them. In a run-to-completion (RTC) pipeline, the poller role h and worker role w are two logical roles of the same thread: $\gamma(h) = \gamma(w)$, their CPU service demands are added, and the shared context consumes exactly one slot. Its event ring is $b_\varnothing$. In a staged pipeline, its producer and worker roles satisfy $\gamma(h) \neq \gamma(w)$ and use a material event ring.

For example, an RTC poller demand of $a_{q\ell}^{H,\mathrm{rx}}$ and worker demand $a_{q\ell}^{W}$ assigned to one x consume one slot on $\eta(x)$ but contribute their sum to processing capacity. Two independent staged threads use distinct x_h, x_w; assigning both to one PU would consume two slots and is forbidden when the ordinary bound is one. RTC therefore never requires increasing a PU multiplicity bound, and increasing such a bound cannot be used to disguise two independent threads as one occupant.

4.3 Hard feasibility constraints

Eligibility predicates A_X encode frontend capability, queue compatibility, administrative CPU sets, available memory, and isolation policy. For every q and d, feasibility includes

$$A_{RT}[r_q, t_q] = 1, \qquad A^d_{QM}[q, \mu^d(q)] = 1, \qquad A^d_{QD}[q, \xi^d(q)] = 1, \tag{3}$$

$$A_{QB}[q, \varphi(q)] = 1, \qquad A_{QA}[q, \zeta(q)] = 1, \qquad A_{QW}[q, \omega(q)] = 1, \tag{4}$$

$$\kappa^d(q) \in \mathcal{I}^d_q, \qquad O^d(q) = O_{qd\kappa^d(q)}, \tag{5}$$

$$\bar{\boldsymbol{\pi}}^d_q = \bar{\boldsymbol{\pi}}_{qd\kappa^d(q)}, \qquad |O^d(q)| = k^d_q, \tag{6}$$

$$A^d_{QH}[q, h] = 1 \quad (h \in O^d(q)), \qquad \sum_{h \in O^d(q)} \bar{\pi}^d_{qh} = 1, \quad \bar{\pi}^d_{qh} \geq 0 \quad (k^d_q > 0), \tag{7}$$

$$A_{LX}[u, \gamma(u)] = 1 \quad (u \text{ active}), \qquad A_{XC}[x, \eta(x)] = 1 \quad (x \text{ active}). \tag{8}$$

The last line applies only to active, controllable physical contexts. For distinct active logical roles u, v, $\gamma(u) = \gamma(v)$ additionally requires the predeclared co-execution predicate $A_{LL}[u, v] = 1$; the RTC equalities and staged inequalities above are mandatory instances. Because γ and η are functions, every shared role and physical context has one consistent realization and PU across all pipelines.

Let U^W_w, U^H_h, U^X_x, and U^C_c be safe service capacities in PU-seconds/s; let $0 < \rho_W, \rho_H, \rho_X, \rho_C < 1$ be utilization guards frozen from calibration. Define the total demand of physical execution context x as

$$A_{x\ell}(\Pi) = \sum_{q:\gamma(\omega(q))=x} a^W_{q\ell} + \sum_d \sum_q \sum_{h \in O^d(q):\gamma(h)=x} \bar{\pi}^d_{qh} a^{H,d}_{q\ell}. \tag{9}$$

Worker-role, I/O-role, physical-context, and PU capacities require

$$\sum_{q:\omega(q)=w} a^W_{q\ell} \leq \rho_W U^W_w \qquad (w \in \mathcal{W}), \tag{10}$$

$$\sum_d \sum_{q:h \in O^d(q)} \bar{\pi}^d_{qh} a^{H,d}_{q\ell} \leq \rho_H U^H_h \qquad (h \in \mathcal{H}), \tag{11}$$

$$A_{x\ell}(\Pi) \leq \rho_X U^X_x \qquad (x \text{ active}), \tag{12}$$

$$\sum_{x:\eta(x)=c} A_{x\ell}(\Pi) \leq \rho_C U^C_c \qquad (c \in \mathcal{C}). \tag{13}$$

Thus RTC roles sharing one x occupy one slot while both terms enter $A_{x\ell}$ and the PU sum. Rates $\lambda^d_{q\ell}$ and service demands $a^W_{q\ell}, a^{H,d}_{q\ell}$ are never used interchangeably: demands are measured service time per packet multiplied by the applicable directional rate, with units recorded in the calibration artifact. With

direction-specific hardware-queue capacities U_r^d and guard ρ_R,

$$\sum_{q:r_q=r} \lambda_{q\ell}^{\mathrm{rx}} \le \rho_R U_r^{\mathrm{rx}} \qquad (r \in \mathcal{R}^{\mathrm{rx}}), \tag{14}$$

$$\sum_{q:t_q=r} \lambda_{q\ell}^{\mathrm{tx}} \le \rho_R U_r^{\mathrm{tx}} \qquad (r \in \mathcal{R}^{\mathrm{tx}}). \tag{15}$$

For every memory-object type $Y \in \{\mathcal{M}^d, \mathcal{D}^d, \mathcal{B}, \mathcal{A}\}$, let $\mathrm{mat}(Y)$ exclude its distinguished null object when one exists. Let b_{qy}^Y be the bytes reserved for q when it selects material object y, and K_y^Y the allocatable capacity. Event rings additionally have required slots g_{qb}, slot capacity G_b, sustainable event rate U_b^B, and guard ρ_B. The null-object reservations are defined as $b_{qb_\varnothing}^B = g_{qb_\varnothing} = 0$ and $b_{qa_\varnothing}^A = 0$. Thus

$$\sum_{q:y_q^Y=y} b_{qy}^Y \le K_y^Y \qquad (y \in \mathrm{mat}(Y)), \tag{16}$$

$$\sum_{q:\varphi(q)=b} g_{qb} \le G_b \qquad (b \in \mathcal{B}^+), \tag{17}$$

$$\sum_{q:\varphi(q)=b} \lambda_{q\ell}^B \le \rho_B U_b^B \qquad (b \in \mathcal{B}^+). \tag{18}$$

Here y_q^Y denotes the relevant component of Π; shared material objects consume the sum of declared reservations rather than being counted once per identity. The null objects are not capacity-bearing resources.

PU multiplicity, SMT, exclusivity, and isolation count physical occupants rather than logical roles. With active set $\mathcal{X}_{\mathrm{act}}(\Pi)$, indicator $z_{xc} = \mathbf{1}[\eta(x) = c]$, physical-core group $K(c) \in \mathcal{K}$, and declared bounds L_c^X and L_k^{SMT},

$$\sum_{x \in \mathcal{X}_{\mathrm{act}}(\Pi)} z_{xc} \le L_c^X, \tag{19}$$

$$\sum_{c:K(c)=k} \mathbf{1}\left[\sum_{x \in \mathcal{X}_{\mathrm{act}}(\Pi)} z_{xc} > 0\right] \le L_k^{\mathrm{SMT}} \qquad (k \in \mathcal{K}). \tag{20}$$

The ordinary isolated-thread profile sets $L_c^X = 1$; RTC co-execution does not change it. Setting $L_k^{\mathrm{SMT}} = 1$ forbids simultaneous use of sibling PUs. Reserved PUs are removed by A_{XC}. A declared role exclusion pair $(u, v) \in \mathcal{E}_{LL}$ requires $c(u) \neq c(v)$, while an independent-thread condition requires $\gamma(u) \neq \gamma(v)$ even if controlled oversubscription is studied separately. More general isolation relations $I_Y(q, q') = 1$ require distinct selected identities for resource type Y; $I_K(q, q') = 1$ requires distinct physical-core groups. These relations cover tenant, control-plane, and experimental isolation without hiding policy in a score. Pool/ring producer–consumer compatibility and frontend-specific RX/TX ownership restrictions are represented by additional Boolean predicates and checked identically.

All capacities, cardinalities, catalog-membership and control-class rules, eligibility rules, sharing rules, and isolation relations are part of $\mathcal{F}(G_s, \ell)$. A requested placement is rejected if any condition fails. For indirect and observed-only ownership, the applicable post-application acceptance set is a run gate rather than an assertion that the planned share was exactly imposed. No finite cost makes an infeasible placement admissible.

4.4 Optimization problem and scope

For a fixed topology snapshot, frontend profile, workload condition, and frozen model artifact, the specification is

$$\Pi^\star \in \arg\min_{\Pi \in \mathcal{F}(G_s, \ell)} J(\Pi; G_s, \ell, \boldsymbol{\alpha}, \boldsymbol{\beta}). \tag{21}$$

The exact minimum defines the comparison target; it does not imply that the heuristic in section 7 finds it. Static attributes include containment, PCIe ancestry, CPU sets, NUMA associations, and unloaded calibrated

paths. Workload-dependent attributes include rates, service demands, resource utilization, and contention estimates from a preceding calibration or observation interval. Measurements from a response interval are never used retroactively to choose that interval's placement.

The primary observable is p99.9 external turnaround latency. J is a relative planning score whose ranking must be validated on held-out placements, not a claim of analytical tail-latency prediction. The model excludes transport redesign, NIC firmware changes, failure recovery, and application-state partitioning beyond declared $\mathcal{A}$ objects. Such requirements may restrict $\mathcal{F}$ but are not optimized here.

5 Typed Hardware-Topology Graph

5.1 Definition

For snapshot s, define

$$G_s = (V, E, \tau_V, \tau_E, a_V, a_E), \tag{22}$$

where V and $E \subseteq V \times V$ are finite vertices and directed edges, $\tau_V : V \to T_V$ and $\tau_E : E \to T_E$ are type functions, and a_V, a_E map objects to typed attribute records. Opposite directed edges may have different attributes; symmetry is never assumed.

Vertex types are grouped as follows:

- *compute*: machine, socket/package, core, PU, and physical execution context/occupancy group;
- *memory/cache*: NUMA node, memory domain, huge-page pool, packet-buffer pool, and cache domain at each relevant level;
- *PCIe/I/O*: root complex or root domain, root port, bridge, switch, endpoint/function, NIC port, RX queue, and TX queue; and
- *software path*: logical poller/IRQ/completion role, runtime-worker role, event bus/ring, and distinguished null role/object where applicable.

Edge types include CONTAINS, SHARES-CACHE, CPU-AFFINITY, REALIZES-ROLE, MEMORY-AFFINITY, PCIE-PARENT, PCIE-ROUTE, DMA-ACCESS, CPU-MEMORY-ACCESS, SOCKET-LINK, OWNS-QUEUE, PRODUCES, and CONSUMES. A REALIZES-ROLE edge maps one or more compatible logical roles to one physical execution context; only those physical contexts consume PU slots. Containment edges form useful hierarchies, but G_s is not restricted to a tree: cache sharing, NUMA distances, alternate routes, and application communication can cross a containment tree. This extends topology-mapping graph formulations [13] with resource and relation types needed for I/O.

Vertex attributes include stable-within-snapshot identity, online state, capacity, eligible CPU set, BDF, queue index, memory policy, page size, location distribution ν_X, and provenance. Edge attributes may include hop or crossing count, direction, negotiated PCIe generation and width, calibrated latency, calibrated bandwidth, counter identity, utilization, confidence, and observation time. Every value carries one of four provenance classes: DISCOVERED, DERIVED, CALIBRATED, or UNKNOWN. $\perp$ does not compare equal to zero and cannot create a preferred path.

5.2 Discovery and normalization

The topology inspector combines complementary sources. hwloc supplies the machine's compute/cache/NUMA hierarchy, CPU sets, distance information where exported, and PCI device ancestry [11]. Linux sysfs supplies PCI devices and parent paths, NUMA hints, online CPU/node lists, network-interface queue directories, MSI-X/IRQ associations where applicable, huge-page inventory, and device link properties exposed by the driver. PCI BDF values join hwloc I/O objects to sysfs and the selected network interface. Linux affinity and NUMA interfaces supply requested masks and policies; independent read-back supplies observed masks and page locations.

Not all graph vertices have a direct kernel object. Hardware queue vertices can be synthesized from driver/frontend metadata and bound to a port/function by a recorded join. A root complex may appear only

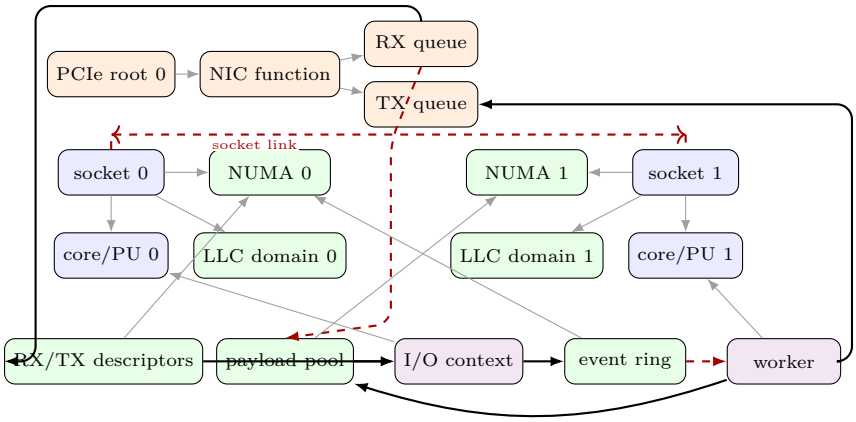


Figure 2: Illustrative typed graph fragment. Gray edges express containment or affinity; solid black and dashed red edges are direction-specific local and cross-socket data/control relations. The graph keeps compute, memory, PCIe, queue, and software objects distinct even when the platform reports proximity between them.

as the boundary above a set of root ports; in that case the builder creates a DERIVED root-domain vertex and records the rule, rather than inventing a physical identifier. Firmware may omit distance, routing, or switch-internal contention. Such attributes remain $\perp$ until a calibration experiment supplies a value.

Normalization enforces invariants before planning:

1. every online PU belongs to exactly one discovered core and one declared package, while NUMA and cache memberships are represented independently;
2. every selected NIC port resolves to one endpoint BDF, and every declared hardware queue resolves to that port;
3. each pool resolves to an allowed set of NUMA nodes and later to an observed page-distribution record;
4. every logical worker/I/O role resolves to a permitted physical execution context, and every active physical context has an eligible CPU set; and
5. each edge attribute includes its source, collection time, unit, and confidence.

Violation stops planning unless a named fallback profile permits the missing fact. A serialized graph includes a schema version, snapshot identifier, host boot identifier, kernel identity, timestamps, source-file digests, and warnings. hwloc XML exports are retained as portable fixtures for off-stand testing, an approach enabled by hwloc's topology serialization [11].

5.3 Path queries and calibrated attributes

The planner does not equate a NUMA distance with all traffic. It issues typed queries: endpoint-to-descriptor and endpoint-to-payload for RX/TX DMA, memory-to-PU for context/worker access, producer-to-consumer for coherence, and PU-to-endpoint for MMIO or doorbells. A path query returns a vector rather than a scalar:

$$\boldsymbol{d}(u, v) = (h_{\text{pcie}}, h_{\text{socket}}, d_{\text{numa}}, \hat{L}, \hat{B}, \hat{X}, c), \tag{23}$$

where the first three entries are structural, $\hat{L}$ and $\hat{B}$ are calibrated latency and bandwidth when available, $\hat{X}$ is observed contention for a workload interval, and c is confidence. Direction-specific values are retained. Hostping's distinction between structural paths and runtime bottlenecks motivates this separation [21]; asymmetry and congestion results motivate avoiding a hop-only order [19, 5].

5.4 Limits of observability

The graph represents what can be supported, not an idealized circuit diagram. Firmware and the operating system can collapse root-complex details, report a NUMA hint of -1, omit queue-to-interrupt data, virtualize PCI functions, or deny performance-counter access. IOMMU routing and DDIO policy may be only partially observable. Dynamic counter attribution can be socket-wide rather than queue-specific. These limits appear in graph provenance and later confidence intervals; they are not repaired by assuming that a device is local to NUMA node 0. The topology snapshot is also invalidated by CPU/NIC hotplug, driver reload, queue reconfiguration, or a changed boot identity.

6 Placement Cost Model

6.1 Complete request/response decomposition

Let $\mathcal{T}_0$ contain unloaded, per-packet path primitives. For pipeline q, the model distinguishes

$$\begin{aligned}\mathcal{T}_0 = \{&\mathrm{rx_payload_dma}, \mathrm{rx_descriptor_dma}, \mathrm{tx_payload_dma}, \mathrm{tx_descriptor_dma},\\ &\mathrm{io_payload}, \mathrm{io_descriptor}, \mathrm{worker_payload}, \mathrm{ring_sync}, \mathrm{application_state},\\ &\mathrm{mmio_doorbell}, \mathrm{cache_ddio}\}.\end{aligned}$$

The four DMA primitives use direction, endpoint, selected payload or descriptor memory, transaction class, and typed PCIe/socket path. The I/O-role terms join $O^d(q)$ and the derived PU $c(h) = \eta(\gamma(h))$ to payload or descriptor memory and weight each role's expected demand by the catalog-derived $\bar{\pi}^d_{qh}$. The worker term joins $c(\omega(q))$ to the selected payload pool. Ring synchronization follows the actual producer, consumer, and $\nu_B(\varphi(q))$; it is identically zero for $b_\varnothing$. Application-state access uses $\zeta(q)$ and is identically zero for $a_\varnothing$. MMIO/doorbell cost follows the responsible execution context's PU-to-endpoint control path. The cache/DDIO term is a calibrated delta under a standardized cache state, not a claim that every platform implements DDIO. For an observed-only shared profile, $\bar{\boldsymbol{\pi}}$ is a frozen workload-dependent planning input and $\widetilde{\boldsymbol{\pi}}$ is a validation observation; neither is presented as an exactly applied share. RxBisect's separation of descriptor ownership, buffer allocation, and packet delivery motivates retaining these primitives instead of representing "the RX queue" by one scalar [28].

For each $t \in \mathcal{T}_0$, let $z^0_{qt}(\Pi, G_s)$ be either a calibrated unloaded quantity or a declared structural proxy, and let $n_{qt\ell}$ be the expected number of such operations per packet. Define

$$\widehat{z}^0_{qt} = \frac{\max\{0, f_t(z^0_{qt}) - f_t(z^{\mathrm{ref}}_t)\}}{s_t}, \qquad s_t > 0, \tag{24}$$

where f_t is a pre-specified monotone transform, z^{ref}_t an unloaded reference, and s_t a scale stored with the model. The quantity $\widehat{z}^0_{qt}$ is dimensionless and nonnegative.

Let $\mathcal{E}_L$ contain capacity-bearing resources: logical worker/I/O roles, physical execution contexts, PUs, memory controllers, socket links, PCIe links/root domains, and cache domains. Their normalized utilization is $u_e(\Pi, \ell)$, with calibrated safe capacity as denominator. A load surcharge is

$$\widehat{\Psi}_e(u_e) = \frac{\max\{0, \psi_e(u_e) - \psi_e(0)\}}{s^L_e}, \tag{25}$$

where ψ_e is nondecreasing on its validated domain and $s^L_e > 0$. Subtracting $\psi_e(0)$ prevents the unloaded path penalty from being counted again as congestion. Let v_{qe} be the nonnegative fraction of q's measured demand attributed to e.

With $\Lambda_\ell = \sum_q \lambda_{q\ell}$, define $r_{q\ell} = \lambda_{q\ell}/\Lambda_\ell$ when $\Lambda_\ell > 0$. For a topology-only plan with no offered load, use

the explicit convention $r_{q\ell} = 1/|\mathcal{Q}|$ and set all load surcharges to zero. Define normalized within-group scores

$$C_{\text{top}}(\Pi) = \sum_{q\in\mathcal{Q}} r_{q\ell} \sum_{t\in\mathcal{T}_0} \alpha_t n_{qt\ell} \widehat{z}^0_{qt}, \qquad \sum_{t\in\mathcal{T}_0} \alpha_t = 1, \tag{26}$$

$$C_{\text{load}}(\Pi) = \sum_{e\in\mathcal{E}_L} \alpha^L_e \Big(\sum_q r_{q\ell} v_{qe}\Big) \widehat{\Psi}_e(u_e) + \alpha^{\text{bal}} \widehat{B}(\Pi, \ell), \qquad \sum_{e\in\mathcal{E}_L} \alpha^L_e + \alpha^{\text{bal}} = 1. \tag{27}$$

All component weights in these simplices are nonnegative. The complete score is

$$J(\Pi; G_s, \ell, \boldsymbol{\alpha}, \boldsymbol{\beta}) = \beta_{\text{top}} C_{\text{top}}(\Pi) + \beta_{\text{load}} C_{\text{load}}(\Pi) + \beta_{\text{unc}} \widehat{U}(\Pi, G_s) + \beta_{\text{chg}} \widehat{R}(\Pi, \Pi_{\text{prev}}), \tag{28}$$

where $\beta_{\text{top}}, \beta_{\text{load}}, \beta_{\text{unc}}, \beta_{\text{chg}} \geq 0$ set the relative scale of the four groups. $\widehat{B}$ is a normalized execution-context imbalance measure, such as the variance plus maximum utilization after centering at the best feasible value. $\widehat{U}$ is an explicitly reported uncertainty surcharge for non-safety-critical unknown attributes. $\widehat{R}$ is zero for static planning and otherwise measures normalized changes in logical owners, execution-context PUs, memory objects, and rings. Safety-critical unknowns and capacity violations remain infeasible rather than receiving $\widehat{U}$.

6.2 Term ownership and avoidance of double counting

Every calibrated observation has one entry in a term-ownership ledger containing physical mechanism, direction, initiator, target, transaction class, base reference, unit, component term, owning group, component weight, and group coefficient. Four rules are enforced.

1. A primitive uses either its calibrated unloaded value or its structural proxy. PCIe crossings, NUMA distance, and measured primitive latency are not simultaneously charged for the same transaction.
2. A resource's load term is the incremental surcharge in equation (25); its unloaded component remains solely in $\widehat{z}^0$.
3. Hardware counters used to calibrate or explain a primitive are not added again as independent costs. A counter becomes an objective feature only in a separately declared model variant that disables the term it replaces; moving a term between groups never duplicates it.
4. Cache/DDIO and other interactions are calibrated as deltas from a standardized base state. A new interaction enters the score only if it is identifiable in training data and improves held-out ranking under the frozen admission rule.

This ledger is part of the reproducibility package. It makes an ablation interpretable: removing a term removes one owned mechanism rather than leaving a duplicate proxy behind.

6.3 Units, monotonicity, and identifiability

Raw latency, bandwidth, transaction, byte, and utilization quantities retain their units in the calibration record; only their normalized forms enter equation (28). All α component weights and β group coefficients are dimensionless. The two within-group simplex constraints make component shares identifiable inside C_{top} and C_{load}. To remove the remaining common-scale non-identifiability, $\beta_{\text{top}} = 1$ is the reference coefficient whenever the topology group is present; the other β values are relative to it. This convention does not resolve feature collinearity.

Topology structure and unloaded calibration determine the values entering C_{top}. $C_{\text{load}}(\Pi, \ell)$ is workload-dependent through demands and utilization. The α and unfixed β coefficients are estimated on calibration/training runs and frozen within a validity domain; they do not change in response to a confirmatory run. β_{unc} prices non-safety-critical missingness under a predeclared uncertainty policy, and β_{chg} is relevant only to the separately declared re-planning treatment.

Each normalized component is expected to be nondecreasing in an adverse path attribute, operation count, utilization, imbalance, uncertainty, or change when other variables and nonnegative coefficients are held fixed. Increasing a group coefficient cannot reduce J. This does not imply that a physically shorter route always has lower total cost: a longer path with spare capacity can outrank a congested local controller [19, 5, 32]. Direction-specific primitives allow RX writes, TX reads, descriptor traffic, and CPU access to respond differently.

DMA path, socket crossings, and interconnect counters can be collinear; worker and I/O demand can also covary with rate. The model report therefore gives design-matrix rank, condition diagnostics, coefficient uncertainty, pairwise feature association, and held-out rank stability after group removal. Collinear terms are combined or one is disabled according to a rule frozen before confirmatory runs. A positive fitted coefficient is not interpreted causally.

6.4 Pre-specified coefficient estimator

The calibration unit is one complete run that has passed every calibration correctness gate. For accepted run r, let

$$y_r = \log \widehat{Q}_{0.999,r}, \tag{29}$$

where $\widehat{Q}$ is the nearest-rank estimator in equation (43), and let x_{rgj} be frozen normalized feature j in group $g \in \mathcal{G} = \{\text{top}, \text{load}, \text{unc}, \text{chg}\}$. Features retain the term-ledger order and contain no confirmatory observations. Calibration is divided into a training set $\mathcal{C}_{\text{tr}}$ and an untouched external validation set $\mathcal{C}_{\text{val}}$ by complete run and placement relation; no packet from one run can enter another split. Condition $c(r)$ identifies the frozen workload/topology condition.

Let $n_{\text{tr}} = |\mathcal{C}_{\text{tr}}| > 0$. The sole coefficient estimator is nonnegative ridge regression. For each positive λ in a finite grid $\mathcal{L}_\lambda$ frozen before response fitting, it solves

$$(\widehat{\boldsymbol{\delta}}_\lambda, \widehat{\boldsymbol{\theta}}_\lambda) = \arg\min_{\boldsymbol{\delta}, \boldsymbol{\theta} \geq 0} \left\{ \frac{1}{n_{\text{tr}}} \sum_{r \in \mathcal{C}_{\text{tr}}} \left(y_r - \delta_{c(r)} - \sum_{g,j} x_{rgj} \theta_{gj} \right)^2 + \lambda \sum_{g,j} \theta_{gj}^2 \right\}. \tag{30}$$

There is one unpenalized condition intercept and no additional global intercept. Each training condition must contain an accepted run. Intercepts absorb condition levels during fitting and evaluation only; they are discarded before placement scoring. The positive ridge term and nonnegativity produce a deterministic convex estimate under the frozen solver precision and feature order; they do not imply causal coefficients.

Let $\rho(r)$ identify the placement relation of run r, let $\mathcal{V}_f$ be validation fold f, and let $\mathcal{T}_f = \mathcal{C}_{\text{tr}} \setminus \mathcal{V}_f$. Folds are condition-stratified while retaining placement-relation blocking: all replicates with one ρ remain in one fold; every condition participating in cross-validation contains at least two independent placement-relation groups; and, for every $r \in \mathcal{V}_f$, at least one accepted $r' \in \mathcal{T}_f$ satisfies $c(r') = c(r)$. Thus $\widehat{\delta}_{c(r),-f}$ is always estimated from training data. If no fold assignment satisfies all three conditions, preprocessing returns Condition-Coverage-Insufficient and calibration terminates with Calibration-Failed; validation observations cannot be moved into training, assigned an invented intercept, or used to select another estimator. The validation loss for an admissible fold is mean squared error in log latency on complete held-out runs,

$$\text{MSE}_f(\lambda) = |\mathcal{V}_f|^{-1} \sum_{r \in \mathcal{V}_f} (y_r - \widehat{\delta}_{c(r),-f} - \boldsymbol{x}_r^\top \widehat{\boldsymbol{\theta}}_{\lambda,-f})^2. \tag{31}$$

The selected value minimizes the fold-size-weighted mean of equation (31); an exact numerical tie selects the larger λ. The model is then refit on all of $\mathcal{C}_{\text{tr}}$. The dataset and model artifacts store a condition-by-placement-relation coverage map, every candidate and accepted fold, fold-admissibility findings, the grid, seeds, feature order, transforms, scales, solver/tolerance identity, coefficient vector, and predictions. Confirmatory data are never used for fitting, feature selection, regularization, or normalization.

Rank and collinearity are evaluated on the training feature matrix after projection away from the condition-indicator space. Exactly equal columns may be merged only when the term-ownership ledger identifies the same physical mechanism; the merged coefficient represents their combined effect and the constituents are marked non-identifiable. The residualized matrix must meet a rank tolerance τ_{rank} and condition-number limit

κ_{design} fixed from numeric-resolution and coefficient-repeatability tests in a performance-blind implementation pilot. If it does not, the last implicated term in a frozen, mechanism-based precedence order is disabled, and the diagnostic is repeated. No latency-significance test determines this order. If this process removes the topology group, leaves a required group without an admitted term, or still fails the diagnostic, calibration terminates with CALIBRATION-FAILED.

Let $s_g = \sum_j \widehat{\theta}_{gj}$. Admission requires $s_{\text{top}} > 0$. The fitted effective coefficients are factored as

$$\alpha_{gj} = \frac{\widehat{\theta}_{gj}}{s_g} \quad (s_g > 0), \qquad \beta_{\text{top}} = 1, \qquad \beta_g = \frac{s_g}{s_{\text{top}}}. \tag{32}$$

Then $\sum_j \alpha_{gj} = 1$ for every enabled group and $\sum_{g,j} x_{rgj}\widehat{\theta}_{gj} = s_{\text{top}} \sum_g \beta_g \sum_j \alpha_{gj} x_{rgj}$, so the score exactly reproduces the fitted placement component up to the irrelevant common factor s_{top}. A group with $s_g = 0$ receives DISABLED-ZERO-SCALE, $\beta_g = 0$, and no artificial simplex vector.

External validation is partitioned before outcomes are revealed. For $\mathcal{C}_{\text{val}}^{\text{seen}} = \{r \in \mathcal{C}_{\text{val}} : c(r) \in c(\mathcal{C}_{\text{tr}})\}$, absolute log-MSE and residuals use the intercept fitted from $\mathcal{C}_{\text{tr}}$, and within-condition Spearman rank correlation, top-choice accuracy with deterministic cost ties, and regret are also reported. For an unseen condition in $\mathcal{C}_{\text{val}}^{\text{new}}$, no fitted intercept exists: absolute prediction MSE and absolute residuals are undefined and must not be computed by estimating an intercept from validation outcomes. Transfer is evaluated only by within-condition rank, top-choice accuracy, and regret over at least two independently admissible placement relations. These transfer metrics cannot by themselves admit the unseen condition to the primary validity domain. Before validation outcomes are revealed, a performance-blind pilot converts timestamp resolution, replicate noise, candidate decision margin, and intended validity-domain coverage into frozen admissibility limits for the applicable loss, rank, top-choice, regret, and extrapolation checks. The artifact is admitted to B3 planning only if all rank/design checks and the predeclared within-condition admission checks for the primary validity domain pass. Failure returns CALIBRATION-FAILED, blocks the confirmatory B3 campaign, and cannot be repaired with confirmatory data or a silently substituted estimator.

6.5 Seven-phase calibration and validation protocol

Calibration precedes and is disjoint from the confirmatory baseline experiment.

1. **Discover and freeze.** Qualify the topology snapshot, frontend, ownership mode, clocks, power state, counters, and candidate resource inventory. Record every unknown and invalidate the phase on a snapshot change.

2. **Calibrate unloaded primitives.** Measure direction-specific endpoint–payload, endpoint–descriptor, context–memory, worker–memory, ring-handoff, state-access, and MMIO paths under low offered load. Vary one typed relation at a time where the topology permits and standardize cache state.

3. **Calibrate demand and capacity.** Estimate per-packet worker/context service time, safe worker/PU capacities, memory, PCIe, and socket-link knees, operation counts, normalization references, and scales without using baseline outcomes.

4. **Calibrate interactions and congestion.** Sweep controlled utilization of each observable capacity domain; estimate $\psi_e(u) - \psi_e(0)$ and cache/DDIO deltas; reject unidentifiable or non-monotone forms outside a predeclared tolerance.

5. **Identify on training units.** Split by complete run and placement/topology relation, never by packet. Construct condition-stratified blocked folds, validate the condition-coverage map before fitting, apply the frozen rank/collinearity rule, fit equation (30), choose λ by the sole minimum-MSE rule over admissible folds, and recover $\boldsymbol{\alpha}, \boldsymbol{\beta}$ through equation (32) without confirmatory outcomes.

6. **Validate on held-out units.** For conditions represented in training, report external log-MSE, residuals, rank agreement, top-choice accuracy, and regret. For unseen-condition transfer, report only within-condition rank, top-choice, and regret, without fitting a validation intercept. A failed artifact receives CALIBRATION-FAILED and cannot plan confirmatory B3; a passing artifact is restricted to its frozen validity domain and is not repaired with confirmatory data.

7. **Freeze and version.** Store the term ledger, transforms, scales, component weights, group coefficients, regularization rule, uncertainty, validity domain, data split, and digest. Precompute sensitivity variants. No coefficient, term, or normalization changes after confirmatory outcomes are revealed.

Performance-contract work demonstrates the value of explicit critical variables while identifying NIC, PCIe, and queueing as omitted end-to-end components [15]. The protocol above measures those components but makes no prediction-accuracy claim before held-out validation.

6.6 Ablations and limitations

Hard constraints remain $\mathcal{F}$ in equation (21). The planned component ablations re-plan after removing, one at a time, RX payload DMA, descriptor DMA, TX DMA, context-memory, worker-memory, ring synchronization, application-state, MMIO, cache/DDIO, individual congestion resources, and imbalance. Group ablations set exactly one of β_{load}, β_{unc}, or β_{chg} to zero while leaving the ownership ledger intact. A mandatory composite contrast compares a *topology-only* planner ($\beta_{\text{load}} = 0$, unloaded primitives, and equal zero-load shares) with the full *load-aware* planner. Sensitivity varies component weights over their simplices, relative group coefficients over their joint calibration uncertainty, normalization scales, regularization choices, and uncertainty surcharges. Null ring/state terms remain zero and are not treated as successful ablations.

The score is not a cycle-accurate simulator. Counters may lack queue attribution; calibrated values can age with firmware, load, or power state; and interactions can violate additivity. These limits are exposed through validity domains, residuals, term ablation, and transfer tests rather than concealed by unmeasured coefficients.

7 Placement Algorithm

7.1 Inputs, outputs, and completion semantics

The planner receives a validated G_s; fixed RX/TX queue pairs; payload, descriptor, ring, state, worker-role, I/O-role, physical-execution-context, and PU inventories; the RTC/staged co-execution predicates; each frozen finite ownership catalog $\mathcal{P}_q^d$ with control class, stable order, validation predicate, and digest; demand and operation counts for condition ℓ; every eligibility, ownership, capacity, SMT, exclusivity, and isolation constraint; a frozen cost-model artifact; an optional logical-mapping restriction such as $\mathcal{R}_0$; an optional previous placement; and deterministic candidate and search budgets. Observed-only shares enter through their frozen catalog proxy and cannot be varied. Random choice is never used for placement ties.

The status vocabulary separates five logically different outcomes.

FEASIBLE — A complete $\Pi \in \mathcal{F}(G_s, \ell)$ has been revalidated and emitted, possibly at a named fallback fidelity level.

SEARCH-LIMITED — Every required candidate list may be fully exposed, but bounded-depth, node-budgeted construction did not find a feasible global assignment. This status makes no infeasibility claim.

CAP-LIMITED — A candidate-generation or exposure cap stopped at least one finite queue-local domain before its enumerator certified exhaustion.

DATA-INCOMPLETE — A safety-critical topology, capability, unit, ownership, or location fact is unknown and no permitted lower-fidelity formulation can be planned safely.

PROVEN-INFEASIBLE — Reserved for an independently checkable certificate from a complete exact feasibility procedure over the original constrained problem.

Algorithm 1 is a bounded heuristic, not such an exact procedure, and never returns PROVEN-INFEASIBLE. Candidate-list exhaustion means only that all queue-local bundles were exposed. Search-budget exhaustion

concerns combinations of those bundles. Neither event, separately or together under bounded backtracking, proves that $\mathcal{F}(G_s, \ell)$ is empty.

7.2 Candidate bundles and safe exposure

For q, let $\mathcal{L}_q = \{\omega(q)\} \cup O^{\mathrm{rx}}(q) \cup O^{\mathrm{tx}}(q)$ be the logical roles referenced by a candidate. Stable raw enumeration constructs

$$b_q = (q, r_q, t_q, \mu^{\mathrm{rx}}, \mu^{\mathrm{tx}}, \xi^{\mathrm{rx}}, \xi^{\mathrm{tx}}, \varphi, \zeta, w, \\ \kappa^{\mathrm{rx}}, O^{\mathrm{rx}}, \bar{\boldsymbol{\pi}}^{\mathrm{rx}}, \kappa^{\mathrm{tx}}, O^{\mathrm{tx}}, \bar{\boldsymbol{\pi}}^{\mathrm{tx}}, \gamma|_{\mathcal{L}_q}, \eta|_{\gamma(\mathcal{L}_q)}). \tag{33}$$

The tuple (q, r_q, t_q) and its endpoint identities are immutable. For each direction, the bundle enumerator first chooses one admissible finite index $\kappa^d \in \mathcal{I}_q^d$ and copies its derived owner set and planning-share vector; it never generates a real-valued share. A bundle includes every referenced logical owner, service share, occupancy group, and PU. Queue-local rejection checks catalog membership and control scope, null-object eligibility, RTC equality or staged inequality of occupancy groups, co-execution compatibility, role/context/PU eligibility, and direction-specific queue compatibility. Global construction later rechecks aggregate capacities and consistency: if two bundles name the same worker role, I/O role, or physical execution context, the shared mapping must agree; material pools and rings accumulate all reservations.

Candidate pruning is deliberately conservative. Bundle b can dominate b' only if they have identical immutable queue identities, selected resource and role identities, ownership-catalog indices and derived shares, occupancy groups, PU assignments, capacity-consumption vectors, conflict/isolation relations, and incidence on every load-bearing resource, while every primitive cost coordinate of b is no greater and at least one is smaller. This state-independent certificate makes them interchangeable in every residual state except for the improved coordinates. A shorter scalar score or a "more local" path is insufficient because global congestion can reverse that order.

Each finite raw domain has a deterministic enumerator and an EXHAUSTED flag. The planner initially exposes at most K unpruned candidates per queue and expands the prefix geometrically after a failed construction. K is a performance control, not a feasibility assumption. If an operator cap stops enumeration before every relevant flag is EXHAUSTED, the result is CAP-LIMITED. If every list is exhausted, its final size is K_{0q}, but failure of the bounded global search is still SEARCH-LIMITED.

7.3 Construction, conflicts, and exact neighborhoods

Pipelines are ordered by decreasing total service demand $a_{q\ell}^W + \sum_d a_{q\ell}^{H,d}$; ties use port BDF, RX queue index, then TX queue index. For each pipeline, construction chooses the feasible exposed candidate with the lowest incremental value of equation (28). Ties compare, in order, maximum projected PU utilization, maximum projected capacity-resource utilization, number of cross-socket primitives, execution-context PU indices, material memory identities, ring identity, worker identity, and owner-role identities.

When pipelines outnumber preferred PUs, sharing is allowed only through declared shared roles or execution contexts and only while every demand, occupancy, ownership, and SMT constraint remains satisfied. RTC roles already share one occupancy group and do not consume an extra slot. When preferred memory is insufficient, the next feasible location-qualified material object is considered; page size, usable capacity, and allocation policy are never changed. On conflict, deterministic backtracking explores at most depth D and at most B_S construction nodes. The failure record contains observed constraints and assignments from searched branches; it is a conflict witness, not a minimal unsatisfiable set or infeasibility certificate.

After a feasible construction, every refinement pass enumerates these neighborhoods completely:

1. replace one pipeline's selected bundle by every exposed generated alternative;
2. for every compatible pair (q, q'), exchange every nonempty union of six atomic assignment units: RX/TX payload pools; RX/TX descriptor objects; event-ring/state objects; worker role; RX ownership-catalog entry; and TX ownership-catalog entry;
3. relocate one active physical execution context x to every eligible PU, updating every logical role and pipeline that references x atomically;

Algorithm 1 Deterministic topology-aware placement

Require: Validated inputs, finite ownership catalogs, workload ℓ, frozen model, initial exposure K, bounds D, B_S, I

Ensure: FEASIBLE, SEARCH-LIMITED, CAP-LIMITED, or DATA-INCOMPLETE

if a safety-critical fact is unknown and no permitted fallback removes its use **then**
    return DATA-INCOMPLETE
end if
for all $q \in \mathcal{Q}$ in stable queue-identity order **do**
    Enumerate queue-local bundles in stable order up to the current exposure
    Enforce null/RTC/staged eligibility; apply only certified state-independent dominance
end for
Order pipelines by decreasing service demand and stable queue identity
repeat
    Construct greedily with exact residual checks and depth-D, node-B_S backtracking
    if a placement is found **then**
        break
    end if
    if some candidate domain is not exhausted and its cap forbids expansion **then**
        return CAP-LIMITED
    end if
    Expand every nonexhausted candidate prefix geometrically and regenerate the bounded search
until a placement is found or every candidate domain is exhausted
if no placement was found **then**
    return SEARCH-LIMITED
end if
for $i = 1$ to I **do**
    Enumerate and score every feasible move in the five declared neighborhoods
    if no move strictly reduces J **then**
        break
    end if
    Apply the best move using the deterministic tie order
end for
Revalidate all hard constraints and emit FEASIBLE, Π, decomposed score, audit, and input digest

4. apply every admissible logical-role-to-execution-context regrouping over existing contexts that preserves the frozen RTC equalities, staged inequalities, co-execution predicates, and resource cardinalities; and
5. relocate one shared material payload pool, descriptor object, event ring, or application-state object to every compatible location-qualified object, updating all users atomically.

An exchange never swaps q, r_q, t_q, port/BDF identity, flow steering, or hardware endpoints. It changes only the selected atomic assignment units; the global γ and η maps remain attached to their role/context identities unless neighborhoods 3 or 4 changes them. Each proposed pair is accepted only after queue-local eligibility and all global memory, worker-role, I/O-role, execution-context, PU, ownership, SMT, exclusivity, and isolation constraints are recomputed for both pipelines and every shared user. Every feasible member is scored. The largest strictly positive reduction is applied; equal reductions use the same global tie order. Refinement stops at a local optimum for these neighborhoods or after I passes, without implying global optimality.

7.4 Complexity and computational status

Let $Q = |\mathcal{Q}|$, $T = |\mathcal{T}_0| + |\mathcal{E}_L|$, and $K_0 = \max_q K_{0q}$ when every queue-local enumerator is exhausted. Let P_q^W denote the number of eligible worker-role execution-context/PU patterns and H_q^d the number of feasible

direction-d catalog-entry/occupancy/PU patterns. Before eligibility filtering,

$$K_{0q} \le |\mathcal{M}^{\mathrm{rx}}||\mathcal{M}^{\mathrm{tx}}||\mathcal{D}^{\mathrm{rx}}||\mathcal{D}^{\mathrm{tx}}||\mathcal{B}||\mathcal{A}||\mathcal{W}|P_q^W H_q^{\mathrm{rx}} H_q^{\mathrm{tx}}, \tag{34}$$

where the distinguished null objects provide cardinality-one options only when eligible and $P_q^W \le XC$ when at most X physical contexts and C PUs per context are eligible for the chosen worker. If at most X physical contexts and C PUs per owner are eligible, then the finite catalog gives the explicit bound

$$H_q^d \le \sum_{i \in \mathcal{I}_q^d} (XC)^{|O_{qdi}|}. \tag{35}$$

For $k_q^d = 0$ this factor is one; for the primary single-owner profile it is at most $|\mathcal{I}_q^d|XC$. Shared ownership can still make enumeration combinatorial through the enumerated owner sets and their context mappings, but no continuous-simplex factor exists. Since every resource inventory, $\mathcal{I}_q^d$, execution-context choice, and PU choice is finite, equations (34) and (35) establish finiteness of each queue-local domain and of their global Cartesian product.

With constant-time typed path queries after preprocessing, exhaustive generation/scoring costs $O(QK_0T)$, direct certified dominance $O(QK_0^2T)$, and sorting $O(QK_0 \log K_0 + Q \log Q)$. A bounded construction explores at most B_S nodes and performs at most $O(QK_0)$ residual checks per root attempt; the depth-only upper bound is $O((QK_0)^{D+1})$, capped operationally by B_S. Let $N_{\mathrm{exch}} \le (2^6 - 1)\binom{Q}{2}$, N_{shared} be the enumerated context/regrouping/material-object moves, and C_J the cost of a complete feasibility/score update. Refinement costs

$$O(IC_J(QK_0 + N_{\mathrm{exch}} + N_{\mathrm{shared}})). \tag{36}$$

The audit reports exposure flags, K_{0q} when known, cap expansions, search nodes/depth, exact neighborhood sizes, time, and peak memory.

The formulation resembles capacitated assignment and topology mapping, but this manuscript supplies no reduction for the complete typed formulation and makes no complexity-class claim. The exact optimization in equation (21) is a specification; Algorithm 1 has finite budgets and no completeness or global-optimality guarantee.

Independent exact-small reference. For an eligible fixture, let $\mathcal{B}_q^{\mathrm{raw}}$ be the complete finite bundle catalog obtained from all resource inventories and every entry of $\mathcal{P}_q^{\mathrm{rx}}$ and $\mathcal{P}_q^{\mathrm{tx}}$, after only syntactic construction. The exact search universe is

$$\mathcal{U}_{\mathrm{exact}} = \prod_{q \in \mathcal{Q}} \mathcal{B}_q^{\mathrm{raw}}, \qquad N_{\mathrm{cart}} = \prod_q |\mathcal{B}_q^{\mathrm{raw}}|. \tag{37}$$

The reference procedure uses the frozen lexicographic bundle order and visits every element of $\mathcal{U}_{\mathrm{exact}}$. It applies no heuristic dominance rule, candidate cap, locality filter, incumbent-derived restriction, or output from Algorithm 1. For each complete combination it reconstructs shared γ and η, tests every condition in $\mathcal{F}(G_s, \ell)$, records every failing constraint class, and evaluates the full J exactly once if feasible. The minimum uses the same declared global tie order as the specification. A nonempty feasible set returns Exact-Feasible, its minimizing placement, objective value, and a complete-search certificate. An empty set returns Proven-Infeasible only after the visited-combination count equals N_{cart} and all finite enumerators certify exhaustion. Interruption, arithmetic failure, or an incomplete enumerator can never produce that status.

Eligibility is decided before search from four frozen implementation-benchmark limits: maximum pipelines Q_{exact}, maximum N_{cart}, peak-memory budget M_{exact}, and wall-time budget T_{exact}. Their numerical values are selected from off-stand completion benchmarks without heuristic or latency outcomes and frozen before the exact campaign. A fixture is eligible only if $Q \le Q_{\mathrm{exact}}$, N_{cart} is within its limit, and conservative preflight memory/time estimates are within M_{exact} and T_{exact}. Otherwise, or if the frozen runtime budget expires before certified exhaustion, the sole result is Exact-Budget-Exceeded; it is neither infeasibility nor a heuristic failure. The audit retains input and catalog digests, all four limits and estimates, N_{cart}, visited and feasible counts, per-constraint rejection counts, incumbent history, best placement and value when one exists, stable tie trace, elapsed time, peak memory, and terminal status.

The exact procedure will be implemented and tested as a separate future component. Minimal hand-constructed fixtures will cover one feasible placement, a tied optimum, each hard-constraint rejection class, globally inconsistent shared mappings, an empty finite universe, and a budget-exceeded case. Expected counts and optima will be checked manually before the procedure is used to assess the heuristic.

Quality measures. Algorithm quality is evaluated independently of end-to-end policy benefit. On exact-eligible fixtures, the study reports objective rank, normalized optimality gap

$$g = \frac{J(\Pi_{\text{heur}}) - J(\Pi^\star)}{\max\{|J(\Pi^\star)|, \epsilon\}}, \qquad \epsilon > 0, \tag{38}$$

and the missed-feasibility rate: the fraction of EXACT-FEASIBLE fixtures for which the heuristic returned SEARCH-LIMITED or CAP-LIMITED. Fixtures with EXACT-BUDGET-EXCEEDED have no known optimum and are excluded from gap and missed-feasibility denominators. This evaluation can expose incompleteness; it does not make the main heuristic exact. A deterministic locality-only heuristic supplies a second comparator. Larger fixtures report score, completion-status frequencies, planning time, peak memory, and scaling without calling the best observed result optimal.

7.5 Incomplete data, fallback, and re-planning

Four placement-fidelity levels are named in advance. FULL requires PCIe ancestry, NUMA associations, queue identities, location-qualified memory, and applicable context control. NUMA disables unavailable PCIe/calibrated primitives but still performs verified ownership, memory, ring, worker, execution-context, and PU mapping. CPU applies verified logical-owner/worker execution placement without claiming memory or I/O locality. DEFAULT delegates the corresponding controls and is the first experimental baseline. Fidelity and completion status are orthogonal: a successful permitted fallback is FEASIBLE with a lower level; otherwise missing safety-critical data yield DATA-INCOMPLETE. SEARCH-LIMITED and CAP-LIMITED outputs contain no plan and are never pooled with FULL.

The primary experiment uses static planning before each run. Optional runtime re-planning may consume only a preceding observation interval, includes $\widehat{R}$ in equation (28), and applies at a declared epoch boundary; it is a separate treatment. Stale snapshots, hotplug, driver reload, read-back mismatch, insufficient huge pages, failed queue ownership, or unavailable privilege trigger rollback to the last verified state and invalidate the run. An unknown is never converted into apparent locality.

8 Automatic-Configuration Design

8.1 Architecture and information flow

The proposed TopoPlace architecture has seven conceptual components. They exchange versioned documents rather than implicit process state; no component implementation is part of this manuscript.

1. The *topology inspector* collects hwloc, sysfs, BDF, CPU/node, queue, IRQ, huge-page, driver, and clock information with provenance and access errors.
2. The *normalized graph builder* validates joins and emits G_s, an inventory of unknown attributes, and a snapshot digest.
3. The optional *calibrator* also consumes a versioned frontend capability record and its finite ownership catalogs, then emits normalized path attributes, scales, weights, uncertainty, and a validity domain.
4. The *placement planner* consumes G_s, a workload contract, frontend capabilities and catalog digests, hard constraints, a reference-mapping restriction when applicable, and calibrated parameters; it emits a complete RX/TX placement, completion status, decomposed score, candidate/search audit, conflict witnesses, and fallback level. A separate exact-small checker consumes the uncapped raw catalogs and never restricts itself using the heuristic output.

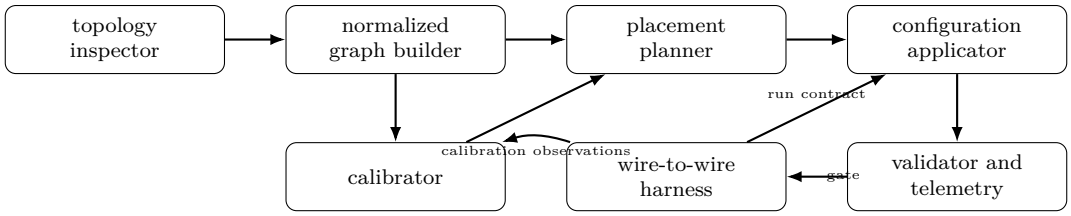


Figure 3: Conceptual information flow. The validator converts applied state and run evidence into an observed-placement gate; desired, applied, and observed records remain distinct and share a topology-snapshot identifier.

5. The *configuration applicator and page initializer* convert a FEASIBLE abstract placement into ordered frontend-specific operations for fresh allocation and faulting, physical-execution-context affinity, payload and descriptor memory policy, huge-page-backed pools, event rings, queue ownership, worker mapping, and IRQ, polling, or completion roles.
6. The *validator and telemetry collector* independently reads back logical-role-to-thread grouping, effective physical-context CPU masks, observed CPUs, page distributions, queue/IRQ ownership, observed service shares, link state, counters, and progress. It emits a pass/fail record rather than editing the plan.
7. The *wire-to-wire harness* schedules traffic, external timestamps, sequence manifests, correctness checks, and run metadata without participating in the SUT placement decision.

Figure 3 shows the contracts. Separating desired, applied, and observed placement is essential: an accepted system call proves only that a request was made. Scheduler defects and complex NUMA behavior motivate independent read-back [22].

8.2 Component contracts

The inspector is read-only. Its output includes raw-source references and errors so that a builder cannot mistake missing privilege for missing hardware. The graph builder accepts only mutually consistent host, boot, and collection identifiers. The calibrator declares the topology, frontend, firmware/driver, clock/power profile, packet-size domain, and workload range in which its parameters are valid.

The planner output is declarative: RX/TX BDF, port, and queue identities; reference or optimized worker mappings; selected finite ownership-catalog indices, derived owner sets and planning shares, their direct/indirect/observed-only control class, and catalog digest; RX/TX payload-pool and descriptor-memory identities with ν_X for material objects; explicit $b_\varnothing$ or $a_\varnothing$ where applicable; worker and I/O roles; each role's physical execution context; every active context's PU; occupancy, service, and capacity ledgers; ordered application actions; completion and fallback status; and verification predicates. Only a FEASIBLE plan reaches the applicator. The applicator records desired and requested values separately, produces a journal entry before each mutation, and stops on the first failed prerequisite. For an observed-only share it records no share-setting action and never states that $\bar{\boldsymbol{\pi}}$ was applied. The validator receives the plan but uses independent observations and records $\widetilde{\boldsymbol{\pi}}$ with coverage and the predeclared acceptance-set result. The benchmark starts only after all mandatory predicates pass.

8.3 Applying and validating placement

CPU placement is requested once per physical execution context through thread-affinity or cpuset mechanisms and verified from the effective mask and repeated observed-CPU samples. RTC poller and worker roles must resolve to the same thread/context ID and consume one slot; staged roles must resolve to distinct IDs and slots. The page initializer creates fresh RX/TX payload, controllable descriptor, material event-ring, and material state allocations for every attempt. It records desired policy, requested policy, initializer identity, allocation/fault sequence, and observed page-to-node distribution as distinct facts. Memory policy is requested before faulting where the baseline requires it; the B0/B1 invariant initializer instead uses the common default

policy and completes faulting before data-plane affinity is applied. A requested policy alone is insufficient. DPDK RX/TX ownership and completion responsibilities are verified against the reference or B3-selected catalog entry using frontend telemetry and hardware statistics. AF_XDP queue identity, UMEM, all four ring types, zero-copy/copy mode, need-wakeup, and polling mode are frozen and read back where supported. IRQ affinity is applied and checked only for an interrupt-driven profile; userspace polling instead validates each poller's physical context, PU, owner set, and observed share. An indirect share must lie in its catalog acceptance set; an observed-only share is reported as an observation, not as successful application of $\bar{\boldsymbol{\pi}}$.

RSS keys and indirection tables, flow rules, RX/TX queue counts, descriptor counts, offloads, MTU, and link state belong to the frontend contract. Queue steering research shows that indirection and load balance are distinct from thread pinning [1]. The validator therefore injects a short, non-scientific mapping probe before the measured run and checks per-queue counters against the expected flow-to-queue relation.

8.4 Privileges, observability, and rollback

Likely privileged operations include userspace-driver binding, queue and RSS configuration, interrupt routing, huge-page reservation, memory-policy changes, CPU isolation, selected physical-page queries, and performance-counter access. The future implementation must enumerate each privilege, use the narrowest mechanism available, and fail closed. The design does not assume that all counters or physical addresses will be accessible.

Before application, the journal captures the reversible prior state: CPU masks, memory policy, RSS/flow configuration, IRQ masks, governor/power settings controlled by the experiment, huge-page allocation, and frontend binding. Rollback occurs in reverse dependency order and is followed by read-back. A failed rollback marks the host dirty and prevents further scientific runs until an operator restores and re-snapshots it. Operations such as firmware changes or reboot-time kernel parameters are outside per-run rollback and belong to testbed provisioning.

Telemetry has explicit attribution limits. Uncore counters may cover a socket rather than a stream; PCIe counters can be absent; page-location sampling can perturb caches; and queue counters can wrap. Each record includes scope, unit, width, sample interval, multiplexing status, and missingness. Counter correlation is explanatory and does not substitute for controlled baseline differences.

9 Experimental Methodology

The methodology is specified before implementation. The companion experimental specification supplies implementation-ready contracts; this section fixes the scientific comparison. Feasibility pilots resolve testbed-dependent quantities but are excluded from all confirmatory estimates.

9.1 Research questions, hypotheses, and sole primary estimand

For each scheduled policy attempt in randomized block j, let $A_{bj} = 1$ only when policy $b \in \{B_0, B_1, B_2, B_3\}$ passes every applicable mandatory performance-blind per-attempt acceptance gate; otherwise $A_{bj} = 0$. Let $E_j^{\mathrm{mem}} = 1$ only when the final B0/B1 page record is Pair-Equivalent. Define pair indicators $A_j^{(1,0)} = A_{B_1 j} A_{B_0 j} E_j^{\mathrm{mem}}$, $A_j^{(2,1)} = A_{B_2 j} A_{B_1 j}$, and $A_j^{(3,2)} = A_{B_3 j} A_{B_2 j}$. The run-level sample p99.9 external turnaround latency $Y_{bj\ell}$ enters a contrast only when the corresponding pair indicator is one. RQ1 estimates CPU affinity (B_1 versus B_0); RQ2 the incremental effect of memory binding (B_2 versus B_1); and RQ3 the incremental effect of the complete mapping (B_3 versus B_2). RQ4 treats packet size, offered load, traffic shape, flow count, queue count, and worker count as effect modifiers. RQ5 evaluates cost-term ablations and explanatory counters. RQ6 evaluates policy-rank stability under a declared workload or topology change.

There is one confirmatory estimand. A single reference condition $\ell^\star$ is frozen after a performance-blind feasibility pilot using the decision rules in the campaign protocol. Let $\mathcal{J}_{32} = \{j : A_j^{(3,2)} = 1\}$ among the blocks scheduled at $\ell^\star$. For $j \in \mathcal{J}_{32}$,

$$D_j = \log\left(\frac{Y_{B_3 j \ell^\star}}{Y_{B_2 j \ell^\star}}\right), \qquad \theta_{32}^{\mathrm{acc}} = \mathrm{E}\Big[D_j \mid A_j^{(3,2)} = 1,\ j \text{ scheduled at } \ell^\star\Big]. \tag{39}$$

The expectation is over accepted independent pairs under the frozen testbed and randomized run-order procedure; $\exp(\theta_{32}^{\mathrm{acc}}) - 1$ is the conditional geometric relative change in p99.9. It is not an unconditional effect over every scheduled block. The directional hypotheses are $H_0 : \theta_{32}^{\mathrm{acc}} \geq 0$ and $H_1 : \theta_{32}^{\mathrm{acc}} < 0$. The confirmatory decision uses a one-sided paired t-test at $\alpha = 0.05$ on the independent accepted-pair values D_j, with the aligned one-sided upper confidence bound for $\theta_{32}^{\mathrm{acc}}$; a two-sided 95% Student interval is also reported for estimation. RQ1 and RQ2 use the analogous accepted-pair sets defined by $A_j^{(1,0)}$ and $A_j^{(2,1)}$. Their effects, other workloads, absolute differences, and all p99.99 results are key secondary or exploratory outcomes and cannot substitute for equation (39).

No superiority claim is made before this test. A latency change accompanied by a different packet path, insufficient goodput, or a failed correctness gate is not evidence for the hypothesis. Operational feasibility is a mandatory secondary outcome: each policy's attempt-acceptance rate and each adjacent pair's completion rate are reported without latency-based selection. A policy is not described as practically beneficial when its latency estimate comes from a small selected subset and it has materially greater planning, application, or validation attrition.

9.2 Required testbed and measurement chain

Confirmatory measurement requires two physical hosts: an isolated traffic-generator/measurement host (TG) and the multi-socket SUT. A one-host loopback or colocated-generator configuration is diagnostic only because it omits or perturbs the physical links and competes for SUT resources. Direct links are preferred; any switch, cable, or optical path is identified, isolated, and invariant.

The primary observation follows this complete chain:

> TG request timestamp and transmit → physical link → SUT NIC ingress → RX descriptor and payload DMA → poll/IRQ progress → optional event-ring handoff → worker and application state → response construction → TX payload/descriptor publication and doorbell → NIC transmit → physical link → TG receive timestamp.

Both primary timestamps are in one TG clock domain, so no inter-host clock subtraction is required. The response is called external wire-to-wire *turnaround* latency, not one-way SUT latency. Constant link and TG contributions are not subtracted from the primary comparison.

Timestamp capability is selected by a frozen decision tree. First preference is an external tap/FPGA or equivalent instrument that timestamps request egress and response ingress in one validated hardware clock. Second is correlated TX/RX hardware timestamps from the TG NIC's same physical hardware clock, provided matching capacity, rate behavior, timestamp loss, and capture-layer semantics pass validation. Third is TG software timestamps; these are diagnostic unless their end-to-end error bound, loss rate, and perturbation are established below a predeclared fraction of the smallest relevant effect. One-way SUT timestamps are secondary and require validated physical-clock synchronization, pre/post offset and drift bounds, and a timestamp layer stated precisely. Merely reporting PTP lock is insufficient.

The clock record contains source, clock identifier, capture boundary, nominal resolution, empirical quantization, conversion, wrap width and extension rule, calibration interval, timestamp loss, and overhead. Pre/post loop or known-delay trials bound bias and drift. A run fails if timestamp ambiguity or the error bound exceeds its frozen gate.

9.3 Environment and invariants

The environment table reports CPU model/stepping/microcode, sockets, dies, cores, PUs, SMT, caches, NUMA nodes/distances/memory population, socket links, firmware and power settings, kernel/boot/isolation configuration, NIC/ports/firmware/driver/BDF/full exposed ancestry, PCIe link state, IOMMU and DDIO state where observable, queues/descriptors/offloads/MTU, huge pages, DPDK or AF_XDP versions/modes, compiler/linker and binary digest, TG and timestamp hardware/software, and counter availability. No NIC model, rate, or counter is assumed before stand qualification.

Within a matched block, the following are identical: source and binary; compiler/linker flags and link order; application logic; frontend profile; NIC firmware/driver/link; queue count and fixed (r_q, t_q) pairs; RSS key, table, hash fields, and flow-to-queue rule; ownership cardinalities and frozen catalog digest; descriptors,

offloads, MTU, burst/batch limits; packet format and response semantics; pool/ring usable capacities and page size; worker-role, I/O-role, and physical-execution-context inventories and counts; RTC/staged grouping rules; safety cpuset; frequency/power/isolation settings; TG trace and exact expected sequence set; timestamp method; phase lengths; telemetry; and acceptance gates. B_0–B_2 also share the topology-agnostic reference mapping $\mathcal{R}_0$; B_3 may change only the controllable finite logical mappings explicitly included in its treatment. Process lifetime, allocation/fault order, and reset procedure are fixed to prevent a later policy from inheriting another policy's pages or queue state [23].

The DPDK profile freezes driver binding, environment abstraction layer parameters, RX/TX setup, mempool and descriptor construction, cache/burst settings, polling/completion mode, and self-reported owners. The AF_XDP profile freezes native/generic and copy/zero-copy modes, UMEM frame layout, RX/TX/fill/completion rings, need-wakeup, busy-poll/interrupt behavior, XDP program, and socket-to-queue binding. Confirmatory comparisons use one frozen primary profile with single-owner queue progress whenever a controllable role exists. A multi-owner profile requires a separate finite ownership catalog and share-validation capability record and remains secondary or a replication until that gate passes; a second frontend profile is likewise a replication, not another baseline.

9.4 Topology-blind initialization and page-equivalence gate

The B0/B1 contrast uses a separate initialization context x_{init} that never participates in the measured packet-processing path. In both policies it runs on the same CPU selected from the initialization cpuset by the smallest stable operating-system CPU identifier after functional exclusions. This rule reads no NIC BDF, PCIe ancestry, NUMA identity or distance, page location, cost term, or latency outcome; it is an experimental invariant, not a locality optimization. Under the ordinary OS memory policy it creates fresh objects in the same stable type/identity order and faults every required page with the same access pattern. Faulting completes before any data-plane physical-execution-context affinity is applied. The process, allocations, and frontend state are fresh for every attempted run, so no policy inherits pages or warmed queue state.

For baseline b, run j, and material object m, let $\widetilde{\nu}_{bjm}(n)$ be the independently observed fraction of covered pages on node n. Desired distribution, requested memory policy, and $\widetilde{\nu}$ are separate records. Each attempt first emits an immutable page-initialization record and independently validates the coverage and internal consistency of its own observed vector. It does not compute a pair distance when the matching policy record is absent; the RQ1 relation is then Pair-Pending, and the attempt may continue through measurement under the fixed randomized schedule. After both matching B0 and B1 records exist, the validator or result aggregator joins mandatory objects and evaluates

$$d_{\text{TV}}(\widetilde{\nu}_{B_0jm}, \widetilde{\nu}_{B_1jm}) = \frac{1}{2} \sum_{n \in \mathcal{N}} |\widetilde{\nu}_{B_0jm}(n) - \widetilde{\nu}_{B_1jm}(n)| \leq \tau_{\text{mem}} \tag{40}$$

for every mandatory material payload, controllable descriptor, event-ring, and state object, with compatible observation coverage. A performance-blind pilot selects τ_{mem} from the page-location method's resolution and repeatability under repeated executions of this invariant initializer; the rule and resulting value are frozen before confirmation and cannot be revised after latency is visible. The final pair status is Pair-Equivalent, Page-Equivalence-Unsupported, or Page-Equivalence-Failed. The latter two cannot enter RQ1, but they do not retroactively invalidate an otherwise accepted attempt for another pre-specified contrast. Pair evaluation uses only immutable page records, never latency; it causes no rerun, substitution, schedule delay, or change to randomized order. Passing this gate establishes measured equivalence within τ_{mem}, not equality of unobserved physical pages.

9.5 Strictly incremental placement policies

Hardware queues do not have CPU masks in the abstract. Their CPU association is induced by the poller, IRQ, or completion role that owns them and by the physical execution context realizing that role. Resource inventories, counts, fixed queue pairs, flow steering, ownership cardinalities, and finite ownership catalogs are common invariants. The topology-agnostic $\mathcal{R}_0$ fixes queue-to-worker identities and catalog indices for B_0–B_2. B_3 may optimize those logical relations only over controllable catalog entries while also selecting physical execution-context PUs and memory; an observed-only share remains its frozen workload-dependent

Policy	Exactly controlled	Explicitly unchanged or delegated
B_0 default	Functional launch inside the common safety cpuset, fixed $\mathcal{R}_0$, and the invariant topology-blind initializer	No experiment-selected data-plane affinity or NUMA policy; fixed queues, RSS, catalogs, RTC/staged semantics, and all other invariants remain common
B_1 CPU only	Exact B0 initialization and page-equivalence gate, plus deterministic affinity for every data-plane physical context	No memory-policy or first-touch change; no change to $\mathcal{R}_0$, queues, RSS, pools, descriptors, rings, state, or sizes
B_2 CPU + memory	Exact B1 execution-context assignments and $\mathcal{R}_0$, plus the CPU-access-share memory rule on fresh allocations	No PCIe-root/path term, ownership-catalog change, load-aware joint re-planning, or non-memory change
B_3 complete	Fresh allocation plus the complete FEASIBLE placement, including a controllable finite ownership entry, occupancy/PUs, and location-qualified objects	Queue identities, RSS, workload, RTC/staged semantics, catalogs, and non-placement invariants remain fixed; fallback or limited-search output is not pooled

Table 2: Strictly incremental policy definitions. A baseline changes only the control added in its row.

proxy and is not a B3 decision. This ownership change is a declared treatment component. In the primary single-owner profile, every non-null direction has $k_q^d = 1$ and share one in all four policies.

B_0 is reproducible rather than "whatever happened": its allowed masks, actual CPUs, default page policy, topology-blind initializer, observed page distributions, queue counters, and ownership observations are archived. If a frontend cannot operate without explicit CPU assignment, its unmodified documented launch mapping is the B_0 default and is reported as such; the study does not call this an operating-system choice.

B_1 orders eligible physical cores by stable OS identifier and assigns active data-plane physical execution contexts round-robin after reserved cores, respecting only functional capacity, SMT, and isolation constraints. Each context is assigned once even when it realizes both RTC poller and worker roles; distinct staged contexts remain distinct occupants. It does not inspect NIC BDF, PCIe ancestry, NUMA distance, page location, or calibrated path cost. This affinity is applied only after the common B0/B1 initializer has completed, and the RQ1 pair is admitted only after the pair-level evaluation in equation (40) returns PAIR-EQUIVALENT. Consequently B1 differs from its paired B0 attempt only by data-plane CPU affinity.

B_2 preserves every B_1 CPU assignment and the complete reference mapping $\mathcal{R}_0$. For each material memory object m, the frozen pipeline contract must list a nonempty set of its actual CPU accessors $\mathcal{X}_m^{\mathrm{cpu}}$ and nonnegative demand shares σ_{mx} summing to one. These may include progress contexts, payload workers, both event-ring endpoints, state users, or another context justified by the frozen packet path. A material object without a justified CPU accessor makes the requested B_2 profile undefined and must be resolved before freezing the campaign; NIC locality is not used as a substitute. With $N(c)$ the NUMA node containing PU c, B_2 selects

$$n_m^{B_2} = \arg\max_{n \in \mathcal{N}} \sum_{x \in \mathcal{X}_m^{\mathrm{cpu}}} \sigma_{mx} \mathbf{1}[N(\eta_{B_1}(x)) = n], \tag{41}$$

using the smallest stable node identifier on a tie. It creates fresh allocations, records desired distribution and requested policy separately, binds or faults m according to the frozen per-object mechanism before applying the preserved B1 data-plane affinity, and verifies the observed distribution. Shares come from operation counts or performance-blind CPU-access calibration, never NIC/PCIe locality or baseline latency. A secondary sensitivity contrasts this aggregate-node rule with a predeclared dominant-accessor rule and demand-share-weighted interleaving when multiple nodes carry material CPU demand; it does not redefine primary B_2. Thus B_2 equals exact B_1 CPU placement plus CPU-relative memory binding and nothing else.

B_3 alone uses the complete topology/load score and may replace $\mathcal{R}_0$ with a feasible optimized $\mathcal{R} = (\omega, \kappa^{\mathrm{rx}}, \kappa^{\mathrm{tx}})$. Three pre-specified secondary mechanism analyses separate ownership and physical placement: full B_3; B_3 with $\mathcal{R} = \mathcal{R}_0$ while physical placement is optimized; and controllable ownership/worker mapping optimized while every other assignment is frozen at its verified B_2 value. The third contrast is executed only if all alternatives remain feasible and preserve packet semantics and offered work. A shared-owner ablation additionally compares frozen finite catalog entries or, for observed-only sharing, strata defined by

the frozen observed-share acceptance range; it never estimates a continuous share choice or claims that an observed share was applied. These analyses and the separate topology-only versus load-aware contrast are not additional levels in the four-baseline ladder.

9.6 Workload matrix and run protocol

Packet sizes include the minimum supported Ethernet frame, justified interior sizes, the standard-MTU boundary, and a jumbo boundary only if the invariant profile enables it. On-wire accounting states whether preamble, inter-packet gap, and frame check sequence are generated/reported. Queue and worker counts span one through the largest value jointly supported by NIC queues and the isolated PU set using a reduced power-of-two design plus the boundary value; feasible $Q < W$, $Q = W$, and $Q > W$ cases exercise sharing/conflict behavior. Flow counts include one, at least one independently steerable flow per RX queue, and the smallest count whose repeated RSS probe meets the frozen occupancy-stability rule.

Absolute rates are determined on the final stand by a feasibility campaign excluded from inference. A common zero-loss capacity is the minimum validated capacity across policies under the invariant path. Low, interior, first-knee, and highest common pre-gate rates are then selected by a pre-specified rate-sweep rule, not by visually favorable p99 differences. Constant-rate traces isolate placement; replayable ON/OFF traces vary burst factor and duration in units of calibrated service and queue-drain time. Uniform and skewed flow demands are included. The trace digest is identical within each block.

Each attempted run follows one auditable dependency order: (1) reset the environment and verify the neutral prior state; (2) create fresh allocations; (3) perform the invariant topology-blind allocation/fault sequence for B0/B1 or the policy-specific allocation/fault sequence for B2/B3; (4) collect and validate the attempt's own immutable page-location observation, leaving a B0/B1 contrast PAIR-PENDING if its mate is not yet available; (5) apply data-plane affinity and the catalog-defined ownership mapping; (6) complete independent read-back of desired, requested, and observed state; (7) execute fixed warm-up; and (8) measure the committed exact transmitted set, drain, post-validate, and cool down. When both matching page records later exist, the validator or aggregator performs the pair-level join and equation (40) without changing this lifecycle. No policy inherits pages, process state, or warmed queues from another. If a frontend requires a different safe order, stand qualification must identify the unavoidable dependency and demonstrate with repeated performance-blind page-distribution observations that the alternate order does not let data-plane affinity alter B0/B1 memory placement; otherwise that frontend profile is unsupported for RQ1.

The pilot determines a conservative warm-up duration exceeding the longest observed stabilization time for throughput, frequency, temperature, and queue occupancy; that duration and tolerances are frozen. Measurement never stops when a confidence interval "looks precise." Cooldown uses fixed time plus a pre-specified environmental gate; a failed gate consumes an attempted-run slot and is recorded.

Policy order is randomized within complete blocks defined by topology snapshot, frontend, trace, packet size, offered rate, queues/workers, and repetition. Runs from different boot/topology identities are not paired. Pilot runs, timestamp qualification, calibration, and confirmatory runs have disjoint identifiers and artifacts.

9.7 Exact packet accounting and metrics

Before transmission, the TG commits an arbitrary finite expected sequence set E_j and its digest; contiguity is not required. For each $s \in E_j$, let R_s be all received candidate responses carrying s, and let R_s^{ok} be those passing packet format, payload/integrity, request-response binding, and timestamp validity. The latency-eligible set is

$$V_j = \{s \in E_j : |R_s| = |R_s^{\mathrm{ok}}| = 1\}, \qquad L_s = t_s^{\mathrm{rx}} - t_s^{\mathrm{tx}} \quad (s \in V_j). \tag{42}$$

The report separately counts missing sequences ($|R_s| = 0$), corrupt/invalid responses ($|R_s| > 0$ but $|R_s^{\mathrm{ok}}| = 0$), duplicates ($|R_s| > 1$), ambiguous matches, and out-of-range sequences. The no-valid-response rate is $1 - |V_j|/|E_j|$; it is not labeled physical loss without the category decomposition. Goodput counts exactly one valid response per expected request. An initial prefix or terminal suffix cannot disappear because E_j is known independently of receipt.

Let $N = |V_j|$ and let $L_{(1)} \le \cdots \le L_{(N)}$ be the one-indexed order statistics, retaining repeated values when timestamps tie. For $N > 0$, the frozen nearest-rank estimator is

$$\widehat{Q}_p = L_{(k_p)}, \qquad k_p = \max\{1, \lceil pN \rceil\}, \qquad 0 \le p \le 1. \tag{43}$$

Thus $p = 0$ returns the minimum, $p = 1$ the maximum, and every reported positive percentile uses $L_{(\lceil pN \rceil)}$ without interpolation. If $N = 0$, every quantile is undefined and the run is invalid. Invalid, duplicate, corrupt, or ambiguous responses never enter V_j and therefore cannot enter the order statistics. Raw tick differences are wrap-extended and converted once before stable numeric sorting; no intermediate rounding is allowed.

For a bounded-resolution histogram with ordered edges $e_0 < \cdots < e_B$, bins are $[e_{b-1}, e_b)$ except the final closed-right bin. The reported percentile is the unique bin interval whose cumulative count first reaches k_p; both interval bounds, N, and k_p are returned rather than an interpolated point. Underflow or overflow containing k_p gives an unbounded interval and is inadmissible for a primary percentile. The latency distribution reports the nearest-rank p50, p90, p99, and p99.9, maximum, interquartile range, median absolute deviation, and a pre-specified jitter statistic, with N beside every percentile. p99.99 is always exploratory in this campaign. Throughput, goodput, missing/corrupt/duplicate/reorder rates, CPU utilization, cycles per valid response, queue occupancy/drops, planner and configuration time, page location, remote-memory indicators, cache events, memory bandwidth, and PCIe/inter-socket counters are secondary or explanatory.

Each run requires $|V_j|(1 - 0.999) \ge 1000$, hence at least 10^6 valid latency observations for p99.9. The TG therefore transmits a fixed $|E_j|$ chosen before confirmatory execution to satisfy this count under the frozen acceptance-loss allowance and the minimum sustained-time rule. Falling short invalidates the run; collection is not extended after inspecting latency. The count rule supports tail resolution but does not make packets independent.

Raw paired timestamps are the preferred artifact. If storage requires a histogram, its schema declares finite bin edges, interval closure, underflow/overflow counts, integer count width, total-count invariant, every nearest-rank index, the bin containing it, the maximum bin width around p99.9, and the resulting interval-censoring bound for every reported percentile. Such a histogram is described as bounded-resolution, not "lossless," unless exact raw values are recoverable. Quantization and bin-width uncertainty are carried into a sensitivity report.

9.8 Fixed sample size and statistical analysis

An independent pilot estimates the standard deviation s_D of paired log ratios and the performance-blind invalid-attempt fraction. Investigators specify a smallest scientifically relevant relative reduction δ_{rel} from the latency budget before examining policy effects and convert it to $\delta^\star = |\log(1 - \delta_{\mathrm{rel}})|$. For desired power $1 - \beta_{\mathrm{II}}$ and one-sided $\alpha = 0.05$, an initial normal-theory planning approximation is

$$n = \left\lceil \left(\frac{z_{1-\alpha} + z_{1-\beta_{\mathrm{II}}}}{\delta^\star / s_D} \right)^2 \right\rceil. \tag{44}$$

The final planned n is the smallest integer at or above this approximation whose one-sample noncentral-t power for standardized effect $-\delta^\star/s_D$ reaches the target, after applying any predeclared conservative variance inflation. The protocol records the latency-budget rationale, pilot s_D, chosen power, approximation, noncentral-t recomputation, rounding, and inflation. A fixed maximum number of attempted blocks is set by inflating n for the pilot invalid fraction. The confirmatory campaign then runs exactly that attempt schedule; it does not add blocks based on an observed effect, p-value, or confidence width. If fewer than n B3/B2 pairs pass performance-blind gates, the study reports reduced power and does not opportunistically continue. If $n_{\mathrm{acc}} < 2$, the variance and t statistic are undefined, so no confirmatory test is reported.

Let $n_{\mathrm{acc}} = |\mathcal{J}_{32}| = \sum_j A_j^{(3,2)}$ be the number of accepted complete B3/B2 pairs from the fixed attempt schedule. The primary estimate is $\bar{D} = n_{\mathrm{acc}}^{-1} \sum_{j \in \mathcal{J}_{32}} D_j$, reported with $\exp(\bar{D}) - 1$. With sample standard deviation s_D^{conf}, the primary test statistic and upper one-sided 95% confidence bound are

$$T = \frac{\bar{D}}{s_D^{\mathrm{conf}} / \sqrt{n_{\mathrm{acc}}}}, \qquad U_{0.95} = \bar{D} + t_{0.95, n_{\mathrm{acc}} - 1} \frac{s_D^{\mathrm{conf}}}{\sqrt{n_{\mathrm{acc}}}}. \tag{45}$$

The one-sided paired t-test rejects H_0 only when $T < -t_{0.95,n_{\mathrm{acc}}-1}$, equivalently when $U_{0.95} < 0$. The report also gives the two-sided 95% Student interval $\bar{D} \pm t_{0.975,n_{\mathrm{acc}}-1} s_D^{\mathrm{conf}}/\sqrt{n_{\mathrm{acc}}}$. These quantities use run pairs, never individual packets.

The test assumes independent blocks in the target campaign, finite variance of D_j, and a sampling distribution of $\bar{D}$ adequately approximated by the Student model. Before unblinding, the analysis plan freezes diagnostics for block independence, a Q–Q display, skewness and tail-weight summaries, maximum studentized residual, and leave-one-pair-out influence. Diagnostics never authorize deletion. If predeclared asymmetry, tail, or influence thresholds are exceeded, the paired t result remains the sole confirmatory rule but is labeled assumption-sensitive; a studentized paired bootstrap-t interval and a median/sign analysis are reported as robustness checks. No alternative is selected because it is more favorable.

Swapping B_2/B_3 labels within pairs tests the sharp null of no pair-specific treatment effect, not the weak conditional mean null $H_0 : \theta_{32}^{\mathrm{acc}} \geq 0$. That label-swapping randomization test is therefore a secondary sensitivity analysis, using exact enumeration when feasible or a frozen-seed Monte Carlo approximation with reported simulation error. A block-resampled mean interval is an additional sensitivity only when $n_{\mathrm{acc}} \geq 30$; no bias-corrected bootstrap is used with few blocks.

Within-run uncertainty of each sample p99.9 is reported separately using moving-block resampling or stationary subsampling over the time-ordered valid sequence stream, with block length selected from a pilot autocorrelation rule. Packets are never treated as independent experimental repetitions. RQ1 and RQ2 paired log-ratio tests form one key-secondary family controlled by Holm's procedure; workload interactions, ablations, RQ5, and RQ6 are labeled secondary/exploratory with their own declared families. Reports give planned and attempted block counts; accepted attempts separately for B0–B3; complete-pair counts for B1/B0, B2/B1, and B3/B2; and policy-specific gate-failure counts and rates. Failure categories distinguish planner, calibration/model domain, application, rollback, placement read-back, page verification, correctness, timestamp, telemetry, and environment, and identify which policy made a block incomplete. Planner/model reporting separately gives the proportions of Search-Limited, Cap-Limited, Data-Incomplete, Calibration-Failed, and application failure using explicit denominators; a campaign-wide calibration failure is one model-admission failure, not duplicated as a latency observation for every block.

No run is removed because its latency is inconvenient. Exclusions are determined without consulting $Y_{bj\ell}$ and are limited to the gates below, a recorded external stand fault, or a missing artifact designated mandatory in advance. All attempts and reasons remain in the index. There are no outcome-dependent reruns, neighboring-run substitutions, or arbitrary latency imputations for failed attempts. The complete-pair primary analysis remains unchanged. Differential attrition is an interpretation limit and a secondary operational-feasibility outcome. Weighting or bounding analyses are exploratory only when their covariates, positivity checks, formulas, and bounds were frozen before unblinding; they never replace the primary test.

9.9 Correctness gates, validity, and reproducibility

A run is admitted only if it passes packet format and integrity; exact E_j accounting; the frozen missing/corrupt/duplicate/reorder thresholds; ring overwrite/lapping detection; producer/consumer progress and complete drain; logical-role-to-physical-context grouping and effective affinity; one-slot RTC and distinct-slot staged occupancy; its own desired/requested/observed location records and page-observation coverage for every material payload, descriptor, ring, and state object; reference or B3-selected finite queue-ownership entry, worker mapping, observed-share acceptance predicate, and RSS mapping; topology digest; clock/timestamp integrity; and mandatory telemetry. The later B0/B1 object-wise pair gate controls RQ1 pair admission, not the already frozen acceptance decision for an otherwise valid individual attempt. Unobservable mandatory page location yields the predeclared unsupported status rather than an assumed pass. A saturation study may intentionally violate a loss threshold but is exploratory and cannot enter equation (39). An attempt is rejected when one of its own gates fails; post hoc "good intervals" are forbidden.

Cost ablations re-plan after removing the owned component or group. Sensitivity covers component and group-level coefficients, topology-only versus load-aware planning, the three ownership/physical-placement mechanism variants, primary single-owner versus qualified finite shared-owner profiles, packet size/rate/burstiness/flow skew, queue/worker counts, SMT, local/cross-socket candidates, normalization, missing information, and one controlled topology change if available. Algorithm quality is evaluated against

the independently complete exact-small and locality-only comparators specified in section 7, including missed-feasibility rate; EXACT-BUDGET-EXCEEDED cases have no certified optimum and do not change the main heuristic's status semantics.

Internal-validity threats include thermal/frequency drift, allocation history, RSS hashing, background work, interrupt leakage, order effects, and policy-dependent planning/application/read-back failure; isolation, fixed lifecycles, monitoring, randomized order, immutable gates, and policy-specific attrition reporting mitigate but do not remove them. Construct validity is limited by turnaround including invariant TG/link time and by imperfect counters; the explicit measurement chain and one clock domain bound interpretation. Conclusion validity is limited by dependent observations, extreme-quantile uncertainty, invalid runs, multiplicity, and selection into accepted pairs. Run-level pairing, fixed sample size, within-run uncertainty, pre-specified families, and operational-feasibility reporting address uncertainty, while the estimand remains explicitly conditional and differential attrition limits generalization. External validity remains bounded by the tested CPU, NIC, firmware, topology, and frontend.

The reproducibility package will contain topology/raw-source snapshots, environment and binary manifests, frontend capability and finite ownership catalogs, calibration and model artifacts including condition-coverage maps, candidate and planner audits, exact-small certificates and budget records, desired, requested, applied, and observed placements, immutable per-attempt page-initialization observations and pair-equivalence records, TG traces and exact E_j sets, raw timestamps or bounded-resolution histograms with nearest-rank indices and containing intervals, clock validation, counters, gates, all planned, attempted, and accepted policy and pair indicators, categorized failures and denominators, the policy causing each incomplete block, exclusion reasons, the frozen attrition-sensitivity specification, the remaining analysis specification, and checksums. No measurement artifact or experimental result exists at the time of writing.

10 Results, Discussion, and Conclusion Placeholder

This section is intentionally not populated. Real experiments must provide the environment/topology table; timestamp qualification; the calibration condition-by-placement-relation coverage map, fold audit, and separate seen-condition absolute and unseen-condition transfer validation records; immutable per-attempt B0/B1 page records and pending/final pair-equivalence states; planned, attempted, and accepted counts for every policy; adjacent-pair counts; policy-specific categorized failure rates and the resulting operational-feasibility assessment; the correctness summary; nearest-rank p50 through p99.9 estimates with sample counts and within-run uncertainty; exploratory p99.99; the sole primary conditional accepted-pair B_3/B_2 mean log-ratio estimate, paired t-test, and aligned confidence bound; key-secondary B_1/B_0 and B_2/B_1 accepted-pair effects; load–latency curves; queue/worker scaling; topology-only, group/component-cost, and ownership-mapping ablations; exact-small-instance heuristic gap and missed-feasibility results with exact-reference terminal statuses; counter associations; sensitivity analyses; and planner/configuration overhead. Only after those data pass the gates in section 9 can the discussion decide the conditional hypothesis, assess differential attrition, state limitations observed on the stand, and draw an evidence-based conclusion. No numerical result or claim of improvement is implied by the model.

References

[1] Tom Barbette, Georgios P. Katsikas, Gerald Q. Maguire Jr., and Dejan Kostić. RSS++: Load and state-aware receive side scaling. In *Proceedings of the 15th International Conference on Emerging Networking Experiments and Technologies*, 2019.

[2] Tom Barbette, Cyril Soldani, and Laurent Mathy. Fast userspace packet processing. In *ACM/IEEE Symposium on Architectures for Networking and Communications Systems*, 2015.

[3] Sergey Blagodurov, Sergey Zhuravlev, Mohammad Dashti, and Alexandra Fedorova. A case for NUMA-aware contention management on multicore systems. In *2011 USENIX Annual Technical Conference*, 2011.

[4] François Broquedis, Jérôme Clet-Ortega, Stéphanie Moreaud, Nathalie Furmento, Brice Goglin, Guillaume Mercier, Samuel Thibault, and Raymond Namyst. hwloc: A generic framework for managing hardware affinities in HPC applications. In *18th Euromicro International Conference on Parallel, Distributed and Network-Based Processing*, 2010.

[5] Mohammad Dashti, Alexandra Fedorova, Justin Funston, Fabien Gaud, Renaud Lachaize, Baptiste Lepers, Vivien Quéma, and Mark Roth. Traffic management: A holistic approach to memory placement on NUMA systems. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2013.

[6] Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.

[7] Killian Castillon du Perron, Dino Lopez Pacheco, and Fabrice Huet. Understanding delays in AF_XDP-based applications. In *2024 IEEE International Conference on Communications (ICC)*, pages 5497–5502, 2024.

[8] Marco Faltelli, Giacomo Belocchi, Francesco Quaglia, and Giuseppe Bianchi. COREC: Concurrent non-blocking single-queue receive driver for low latency networking. *arXiv preprint arXiv:2401.12815*, 2024.

[9] Alireza Farshin, Amir Roozbeh, Gerald Q. Maguire Jr., and Dejan Kostić. Reexamining direct cache access to optimize I/O-intensive applications for multi-hundred-gigabit networks. In *2020 USENIX Annual Technical Conference*, 2020.

[10] Joshua Fried, Zhenyuan Ruan, Amy Ousterhout, and Adam Belay. Caladan: Mitigating interference at microsecond timescales. In *14th USENIX Symposium on Operating Systems Design and Implementation*, 2020.

[11] Brice Goglin. Managing the topology of heterogeneous cluster nodes with hardware locality (hwloc). In *International Conference on High Performance Computing and Simulation*, 2014.

[12] Pilar González-Férez and Angelos Bilas. NUMA impact on network storage protocols over high-speed raw ethernet. In *Second International Workshop on Sustainable Ultrascale Computing Systems*, pages 83–94, 2015.

[13] Torsten Hoefler and Marc Snir. Generic topology mapping strategies for large-scale parallel architectures. In *Proceedings of the International Conference on Supercomputing*, 2011.

[14] Toke Høiland-Jørgensen, Jesper Dangaard Brouer, Daniel Borkmann, John Fastabend, Tom Herbert, David Ahern, and David Miller. The eXpress Data Path: Fast programmable packet processing in the operating system kernel. In *Proceedings of the 14th International Conference on Emerging Networking Experiments and Technologies*, 2018.

[15] Rishabh Iyer, Luis Pedrosa, Arseniy Zaostrovnykh, Solal Pirelli, Katerina Argyraki, and George Candea. Performance contracts for software network functions. In *16th USENIX Symposium on Networked Systems Design and Implementation*, 2019.

[16] Patrick Jahnke, Vincent Riesop, Pierre-Louis Roman, Pavel Chuprikov, and Patrick Eugster. Live in the express lane. In *2021 USENIX Annual Technical Conference*, pages 581–595, 2021.

[17] Emmanuel Jeannot, Guillaume Mercier, and François Tessier. Process placement in multicore clusters: Algorithmic issues and practical techniques. *IEEE Transactions on Parallel and Distributed Systems*, 25(4):993–1002, 2014.

[18] Kostis Kaffes, Timothy Chong, Jack Tigar Humphries, Adam Belay, David Mazières, and Christos Kozyrakis. Shinjuku: Preemptive scheduling for microsecond-scale tail latency. In *16th USENIX Symposium on Networked Systems Design and Implementation*, 2019.

[19] Baptiste Lepers, Vivien Quéma, and Alexandra Fedorova. Thread and memory placement on NUMA systems: Asymmetry matters. In *2015 USENIX Annual Technical Conference*, 2015.

[20] Hyeontaek Lim, Dongsu Han, David G. Andersen, and Michael Kaminsky. MICA: A holistic approach to fast in-memory key-value storage. In *11th USENIX Symposium on Networked Systems Design and Implementation*, 2014.

[21] Kefei Liu, Zhuo Jiang, Jiao Zhang, Haoran Wei, Xiaolong Zhong, Lizhuang Tan, Tian Pan, and Tao Huang. Hostping: Diagnosing intra-host network bottlenecks in real time. In *20th USENIX Symposium on Networked Systems Design and Implementation*, 2023.

[22] Jean-Pierre Lozi, Baptiste Lepers, Justin Funston, Fabien Gaud, Vivien Quéma, and Alexandra Fedorova. The linux scheduler: A decade of wasted cores. In *Proceedings of the Eleventh European Conference on Computer Systems*, 2016.

[23] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the Fourteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2009.

[24] Rolf Neugebauer, Gianni Antichi, José Fernando Zazo, Yury Audzevich, Sergio López-Buedo, and Andrew W. Moore. Understanding PCIe performance for end host networking. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, pages 327–341, 2018.

[25] Amy Ousterhout, Joshua Fried, Jonathan Behrens, Adam Belay, and Hari Balakrishnan. Shenango: Achieving high CPU efficiency for latency-sensitive datacenter workloads. In *16th USENIX Symposium on Networked Systems Design and Implementation*, 2019.

[26] Francisco Pereira, Fernando M. V. Ramos, and Luis Pedrosa. Automatic parallelization of software network functions. In *21st USENIX Symposium on Networked Systems Design and Implementation*, 2024.

[27] Boris Pismenny, Adam Morrison, and Dan Tsafrir. ShRing: Networking with shared receive rings. In *17th USENIX Symposium on Operating Systems Design and Implementation*, pages 949–968, 2023.

[28] Boris Pismenny, Adam Morrison, and Dan Tsafrir. Disentangling the dual role of NIC receive rings. In *19th USENIX Symposium on Operating Systems Design and Implementation*, pages 651–669, 2025.

[29] George Prekas, Marios Kogias, and Edouard Bugnion. ZygOS: Achieving low tail latency for microsecond-scale networked tasks. In *Proceedings of the 26th Symposium on Operating Systems Principles*, 2017.

[30] Henry Qin, Qian Li, Jacqueline Speiser, Peter Kraft, and John Ousterhout. Arachne: Core-aware thread management. In *13th USENIX Symposium on Operating Systems Design and Implementation*, 2018.

[31] Hugo Sadok, Nirav Atre, Zhipeng Zhao, Daniel S. Berger, James C. Hoe, Aurojit Panda, Justine Sherry, and Ren Wang. Ensō: A streaming interface for NIC-application communication. In *17th USENIX Symposium on Operating Systems Design and Implementation*, 2023.

[32] Mohammad Siavashi, Alireza Sanaee, Mohsen Sharifi, and Gianni Antichi. Phoenix: A novel technique for performance-aware orchestration of thread and page table placement in NUMA systems. *SN Computer Science*, 7, 2026. Article 548.

[33] Igor Smolyar, Alex Markuze, Boris Pismenny, Haggai Eran, Gerd Zellweger, Austin Bolen, Liran Liss, Adam Morrison, and Dan Tsafrir. IOctopus: Outsmarting nonuniform DMA. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 101–115, 2020.

[34] Vipin Varghese. [PATCH v4 0/4] Introduce Topology NUMA grouping for lcores. DPDK development mailing list, 5 November 2024. Primary archive: `https://mails.dpdk.org/archives/dev/2024-November/308191.html`, 2024. Follow-up review discussion dated 17 March 2025: `https://mails.dpdk.org/archives/dev/2025-March/316021.html`.

[35] Qiongwen Xu, Sebastiano Miano, Xiangyu Gao, Tao Wang, Adithya Murugadass, Songyuan Zhang, Anirudh Sivaraman, Gianni Antichi, and Srinivas Narayana. State–compute replication: Parallelizing high-speed stateful packet processing. In *22nd USENIX Symposium on Networked Systems Design and Implementation*, pages 937–957, 2025.